HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO HỌC PHẦN

ĐỀ TÀI: Xây dựng hệ thống IoT giám sát và điều khiển thiết bị

Học phần	**: IoT và Ứng dụng**
Giảng viên hướng dẫn	**: Nguyễn Quốc Uy**
Sinh viên thực hiện	**: Trần Đức Quân**
Mã sinh viên	**: B23DCCN684**

Hà Nội – 2026

LỜI CẢM ƠN

Lời đầu tiên, em xin gửi lời cảm ơn sâu sắc tới TS. Nguyễn Quốc Uy giảng viên Khoa Công nghệ thông tin, Học viện Công nghệ Bưu chính Viễn thông, người đã luôn tận tâm, nhiệt tình hướng dẫn và chỉ bảo em trong suốt quá trình thực hiện đề tài này. Những lời khuyên, sự chỉ dẫn quý báu của thầy đã giúp em vượt qua rất nhiều khó khăn và hoàn thiện đề tài một cách tốt nhất.

Tuy nhiên, do vẫn còn nhiều thiếu sót về kiến thức, kỹ năng cũng như kinh nghiệm thực tế, bài báo cáo của em không thể tránh khỏi những sai sót. Em rất mong nhận được sự đóng góp, phê bình từ quý thầy cô để có thể hoàn thiện hơn trong những nghiên cứu và dự án sau này.

Quá trình thực hiện đề tài lần này đã giúp em nhận ra được tầm quan trọng của sự tự giác và chủ động trong công việc. Em hiểu rằng để đạt được kết quả tốt, cần phải tự mình tìm hiểu, nghiên cứu và không ngừng rèn luyện những kỹ năng còn thiếu sót.

Một lần nữa, em xin bày tỏ lòng biết ơn sâu sắc và sự cảm kích đối với sự tận tâm, nhiệt huyết của thầy, đã giúp em vượt qua khó khăn và tiến bộ trong quá trình học tập.

Em xin chân thành cảm ơn!

Mục lục

LỜI CẢM ƠN....2
Mục lục....3
Danh mục hình ảnh....4
Danh mục bảng biểu....5
Chương I: Mô tả chung đề tài....6
1.1. Giới thiệu chung....6
1.2. Mục tiêu nghiên cứu....6
1.3. Đối tượng và phạm vi nghiên cứu....7
1.4. Tổng kết chương....7
Chương II: Thiết kế tổng quan....9
2.1. MQTT là gì?....9
2.2. Kiến trúc hệ thống....9
2.3. Thiết bị phần cứng....10
2.4. Công nghệ sử dụng....13
2.5. Tổng kết chương....14
Chương III: Thiết kế chi tiết....16
3.1. Usercase01....16
3.2. Usercase02....20
3.3. Usercase03....24
3.4. Usercase04....28
3.5. Usercase05....32
3.6. ERD....34
3.7. Danh sách topic....34
3.8. Tổng kết chương....35
Chương IV: Triển khai hệ thống và kết quả....37
4.1. Lắp ráp phần cứng....37
4.2. Triển khai phần mềm nhúng trên ESP32....38
4.3. Triển khai Backend và cơ sở dữ liệu....39
4.4. Triển khai giao diện người dùng Frontrend....40
4.5. Kiểm thử và đánh giá kết quả....44
4.6. Tổng kết chương....51
Chương V: Tổng kết và đánh giá....52
5.1. Kết quả đạt được....52
5.2. Hạn chế của hệ thống....53
5.3. Hướng phát triển trong tương lai....53
TÀI LIỆU THAM KHẢO....55

Danh mục hình ảnh

Hình 2.1 Mô hình tổng quan hệ thống.....9
Hình 2.2 Board ESP32 DevKit.....10
Hình 2.3 Cảm biến ánh sáng/Quang trở.....11
Hình 2.4 Cảm biến DHT11.....12
Hình 2.5 Đèn led 5mm.....12
Hình 2.6 Dây nối mạch.....13
Hình 3.1 sequence diagram dashboard.....17
Hình 3.2 Giao diện màn dashboard.....19
Hình 3.3 Sequence diagram sensor_data.....22
Hình 3.4 Giao diện màn sensor_data.....23
Hình 3.5 Sequence diagram active_history.....26
Hình 3.6 Giao diện màn active_history.....27
Hình 3.7 Sequence diagram profile.....29
Hình 3.8 Giao diện màn profile.....30
Hình 3.9 Sequence diagram chức năng tự ghi dữ liệu cảm biến.....33
Hình 3.10 Sơ đồ ERD.....34
Hình 4.1 Mô hình thực tế sau khi lắp ráp.....37
Hình 4.2 Kết quả giao diện trang Dashboard.....39
Hình 4.3 Kết quả giao diện trang Data sensor.....40
Hình 4.4 Kết quả giao diện trang Active History.....41
Hình 4.5 Kết quả giao diện trang Profile.....42
Hình 4.6 Kết quả kiểm thử lấy 5 dữ liệu mới nhất.....43
Hình 4.7 Kết quả kiểm thử lấy danh sách thiết bị.....44
Hình 4.8 Kết quả kiểm thử bật thiết bị khi ngắt kết nối ESP32.....45
Hình 4.9 Kết quả kiểm thử lấy lọc dữ liệu nhiệt độ > 30.....46
Hình 4.10 Kết quả kiểm thử đổi xóa thông tin tên.....47

Danh mục bảng biểu

Bảng 3.1 Danh sách topic..... 35
Bảng 4.1 Kết nối các chân mạch.....36
Bảng 4.2 Bảng kết quả kiểm thử tích hợp toàn hệ thống.....49

Chương I: Mô tả chung đề tài

1.1. Giới thiệu chung

Trong bối cảnh chuyển đổi số và Cách mạng Công nghiệp 4.0, Internet of Things (IoT) đã trở thành nền tảng công nghệ nòng cốt, kết nối vạn vật qua Internet và kiến tạo nên các mô hình hiện đại như "nhà thông minh", "thành phố thông minh" hay "nông nghiệp thông minh". Việc ứng dụng hệ thống IoT không chỉ giúp tự động hóa các hoạt động thường nhật mà còn tối ưu hóa quy trình quản lý, giám sát không gian sống.

Xuất phát từ nhu cầu thực tiễn đó, đề tài "Hệ thống IoT Dashboard giám sát và điều khiển thiết bị" được thực hiện nhằm xây dựng một mô hình phần mềm trực quan, giả lập lại toàn bộ kiến trúc và luồng hoạt động của một hệ thống nhà thông minh. Thông qua nền tảng Web, hệ thống cho phép người dùng:

- Giám sát thời gian thực: Cập nhật liên tục các thông số môi trường (nhiệt độ, độ ẩm, cường độ ánh sáng) trên giao diện.
- Điều khiển từ xa: Thao tác bật/tắt và điều chỉnh các thiết bị (đèn, quạt,...) chỉ với vài cú click chuột.
- Quản lý dữ liệu: Tra cứu nhật ký hoạt động của thiết bị và diễn biến của các chỉ số môi trường qua từng mốc thời gian.

Sản phẩm đóng vai trò như một mô hình thu nhỏ của hệ thống IoT thực tế. Mục đích của đề tài không chỉ là minh họa sự tiện nghi, an toàn và hiện đại của công nghệ Smart Home, mà còn tạo ra một bộ khung kiến trúc phần mềm linh hoạt, có khả năng mở rộng để ứng dụng vào các bài toán quản lý thực tế như tòa nhà văn phòng, phòng học, nhà kính hay không gian nhà xưởng.

1.2. Mục tiêu nghiên cứu

Đề tài hướng tới việc giải quyết các bài toán về luồng dữ liệu và trải nghiệm người dùng với các mục tiêu cụ thể sau:

1. Xây dựng hệ thống IoT: Hoàn thiện một luồng hệ thống khép kín từ việc cảm biến đọc dữ liệu môi trường, truyền tải qua giao thức mạng đến việc tiếp nhận và xử lý lệnh điều khiển thiết bị theo thời gian thực.
2. Phát triển giao diện trực quan (Dashboard): Thiết kế Web UI/UX thân thiện, hiện đại, tích hợp các biểu đồ phân tích và bảng điều khiển trung tâm giúp người dùng dễ dàng theo dõi số liệu và thao tác.
3. Lưu trữ và toàn vẹn dữ liệu: Xây dựng cơ sở dữ liệu tối ưu để lưu trữ khối lượng lớn dữ liệu cảm biến theo thời gian (time-series) và lịch sử thao tác thiết bị, phục vụ cho công tác kiểm toán, phân tích và thống kê sau này.

4. Đảm bảo hiệu năng và độ ổn định: Hệ thống cần duy trì khả năng phản hồi nhanh (độ trễ thấp), cập nhật trạng thái thiết bị chính xác và có cơ chế xử lý ngoại lệ an toàn ngay cả khi luồng dữ liệu gặp sự cố gián đoạn.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Để hiện thực hóa các mục tiêu đã đề ra, đề tài tập trung nghiên cứu và làm rõ các đối tượng sau:

- Mô hình kiến trúc tổng thể và luồng xử lý dữ liệu khép kín của một hệ thống Internet of Things (IoT) ứng dụng trong nhà thông minh.
- Các giao thức truyền thông mạng tối ưu cho hệ thống thời gian thực (Real-time), đặc biệt là sự kết hợp giữa MQTT (giao tiếp thiết bị - máy chủ), WebSocket (đẩy dữ liệu máy chủ - trình duyệt) và RESTful API.
- Công nghệ thiết kế và lập trình Web SPA (Single-Page Application) để xây dựng Dashboard giám sát trực quan, kết hợp với các kỹ thuật trực quan hóa dữ liệu (Data Visualization) qua biểu đồ.
- Kỹ thuật thiết kế cơ sở dữ liệu quan hệ để xử lý và lưu trữ hiệu quả chuỗi dữ liệu thời gian (time-series data) từ cảm biến và lịch sử thao tác hệ thống.

Phạm vi nghiên cứu: Dựa trên nguồn lực và thời gian cho phép, đề tài được giới hạn trong các phạm vi cụ thể nhằm đảm bảo tính khả thi và độ hoàn thiện của sản phẩm:

- Về mặt chức năng: Hệ thống tập trung giải quyết 3 bài toán cốt lõi: Giám sát thông số môi trường (nhiệt độ, độ ẩm, cường độ ánh sáng); Điều khiển trạng thái bật/tắt của thiết bị (đèn, quạt...); và Tra cứu/lọc dữ liệu lịch sử hoạt động. Đề tài chưa đi sâu vào việc phát triển các kịch bản tự động hóa phức tạp (Automation Rules) hay tích hợp các mô hình Trí tuệ nhân tạo (AI) để phân tích thói quen người dùng.
- Về mặt phần cứng: Đề tài tiến hành xây dựng mô hình trạm trung tâm và các node mạng ở quy mô thử nghiệm phòng thí nghiệm (Lab-scale). Hệ thống sử dụng các linh kiện phổ thông, tối ưu chi phí như vi điều khiển ESP32, cảm biến DHT11, cảm biến quang trở và các module Relay/LED giả lập thiết bị điện.
- Về nền tảng triển khai: Ứng dụng quản lý được phát triển độc quyền dưới dạng Ứng dụng Web (Web Dashboard) đáp ứng tốt việc hiển thị trên trình duyệt máy tính. Đề tài chưa mở rộng sang việc phát triển ứng dụng cài đặt gốc trên thiết bị di động (Native Mobile App như Android hay iOS)..

1.4. Tổng kết chương

Trong chương I, đề tài đã trình bày tổng quan về hệ thống IoT Dashboard giám sát và điều khiển thiết bị, làm rõ bối cảnh hình thành, ý nghĩa thực tiễn và các mục

tiêu nghiên cứu cần đạt được. Hệ thống hướng đến việc ứng dụng công nghệ IoT trong giám sát các thông số môi trường, điều khiển thiết bị từ xa, quản lý dữ liệu và theo dõi lịch sử hoạt động thông qua giao diện Web trực quan.

Bên cạnh đó, chương đã xác định các yêu cầu trọng tâm về khả năng cập nhật dữ liệu theo thời gian thực, tính toàn vẹn của dữ liệu, hiệu năng xử lý và độ ổn định của hệ thống. Đây là cơ sở để lựa chọn công nghệ, thiết kế kiến trúc hệ thống trong các chương tiếp theo.

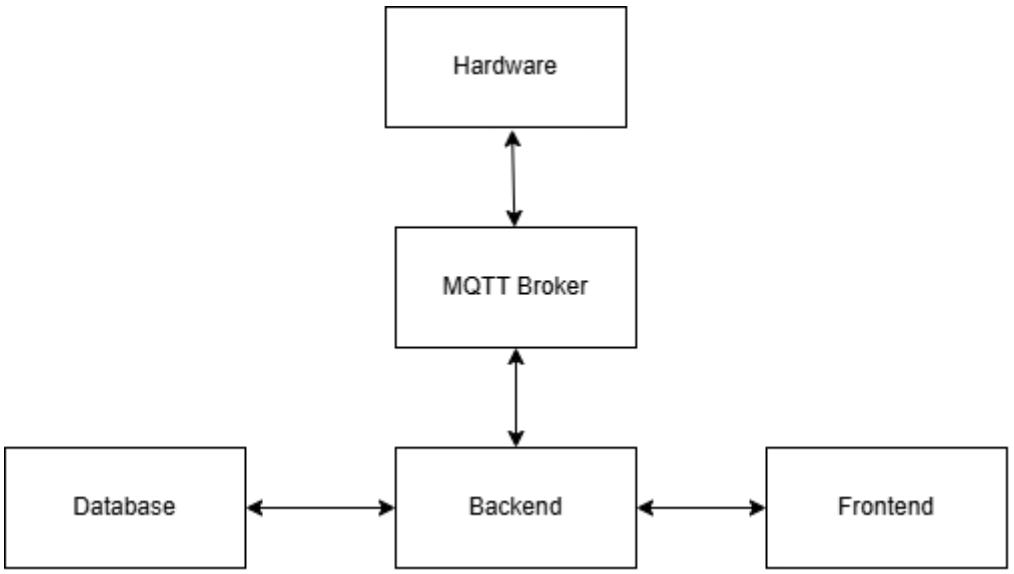
Chương II: Thiết kế tổng quan

2.1. MQTT là gì?

MQTT (Message Queuing Telemetry Transport) là giao thức truyền thông điệp (message) theo mô hình publish/subscribe (cung cấp / thuê bao), được sử dụng cho các thiết bị IoT với băng thông thấp, độ tin cậy cao và khả năng được sử dụng trong mạng lưới không ổn định.

MQTT broker là "trung tâm điều phối" theo mô hình publish/subscribe: thiết bị publish dữ liệu lên một topic, broker nhận message đó rồi phân phối tới mọi client đã subscribe topic tương ứng. Các thiết bị không cần biết địa chỉ của nhau - chỉ cần cùng "nói chuyện" qua broker, nên rất phù hợp cho hệ thống nhiều cảm biến/thiết bị nhỏ, kết nối không ổn định.

2.2. Kiến trúc hệ thống



Hình 2.1 Mô hình tổng quan hệ thống

Sơ đồ kiến trúc hệ thống (Hình 2.1), hệ thống được thiết kế theo mô hình phân tán với 5 thành phần chính có liên kết truyền thông hai chiều. Trong đó, Backend đóng vai trò là lõi xử lý trung tâm, trực tiếp điều phối luồng thông tin giữa Cơ sở dữ liệu (Database), Giao diện người dùng (Frontend) và Khối trung gian MQTT Broker.

Luồng hoạt động của kiến trúc hệ thống được chia thành hai quá trình giao tiếp cốt lõi:

Luồng thu thập và trực quan hóa dữ liệu:

- Khối Hardware (các vi điều khiển, cảm biến) thực hiện việc đo lường thông số môi trường thực tế và đóng gói thành các bản tin (message) để gửi (publish) lên MQTT Broker.

- MQTT Broker đóng vai trò là "trung tâm điều phối", tiếp nhận thông điệp từ phần cứng và tự động phân phối tới Backend (ứng dụng đang đóng vai trò là client subscribe các topic tương ứng).
- Khi Backend nhận được dữ liệu, nó sẽ thực hiện hai tác vụ song song: đẩy dữ liệu vào Database để lưu trữ lịch sử lâu dài, đồng thời chuyển tiếp luồng dữ liệu này lên Frontend thông qua Websocket để người dùng giám sát theo thời gian thực.

Luồng điều khiển thiết bị:

- Người dùng thao tác bật/tắt thiết bị trên màn hình Frontend, trình duyệt sẽ gửi yêu cầu điều khiển về cho khối Backend.
- Backend tiếp nhận yêu cầu, cập nhật trạng thái thao tác vào Database để lưu vết lịch sử hoạt động, sau đó truyền lệnh điều khiển tới MQTT Broker.
- MQTT Broker tiếp tục phân phối (publish) lệnh này xuống khối Hardware. Khối phần cứng tiếp nhận, giải mã lệnh và thực thi đóng/ngắt các cơ cấu chấp hành (đèn, quạt...).

Kiến trúc tách rời này tận dụng được điểm mạnh của mô hình publish/subscribe, cho phép Hardware và Backend có thể giao tiếp, "nói chuyện" với nhau một cách tin cậy thông qua MQTT Broker mà không cần phải định tuyến địa chỉ IP trực tiếp của nhau. Cách tiếp cận này giúp toàn bộ hệ thống hoạt động ổn định, tiết kiệm băng thông và dễ dàng duy trì kết nối ngay cả trong điều kiện mạng lưới IoT không ổn định.

2.3. Thiết bị phần cứng

a. Mạch điều khiển trung tâm

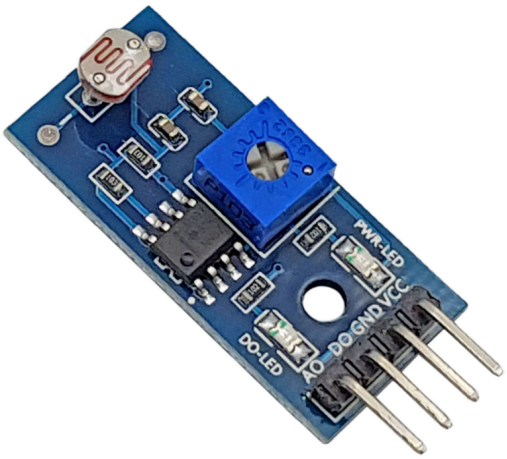


Hình 2.2 Board ESP32 DevKit

Tên: ESP32-WROOM-32 DevKit (NodeMCU-32S)
Công dụng: Vi điều khiển tích hợp Wi-Fi + Bluetooth, dùng làm bộ não xử lý trung tâm cho các dự án IoT, thu thập dữ liệu từ cảm biến và gửi lên Internet
Thông số:

- Chip: ESP32-WROOM-32 (lõi kép Tensilica Xtensa LX6, tốc độ lên đến 240MHz)
- Wi-Fi 802.11 b/g/n, Bluetooth 4.2 (Classic + BLE)
- Bộ nhớ Flash: thường 4MB
- Điện áp cấp nguồn: 5V qua cổng USB-C, hoạt động logic 3.3V
- Số chân GPIO: ~30 chân (D0–D33, VP, VN, EN, TX, RX...)
- Có nút nhấn EN (reset) và BOOT
- Cổng kết nối: USB-C (nạp code + cấp nguồn)

b. Cảm biến ánh sáng



Hình 2.3 Cảm biến ánh sáng/Quang trở

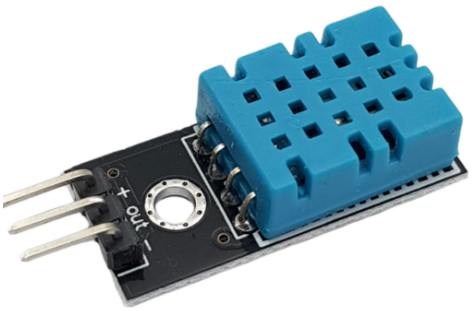
Tên đầy đủ: Module cảm biến ánh sáng dùng quang trở (LDR) kết hợp IC so sánh LM393

Công dụng: Phát hiện cường độ ánh sáng môi trường.

Thông số:

- Điện áp hoạt động: 3.3V – 5V DC
- Output: DO (Digital Output – tín hiệu số HIGH/LOW, có biến trở chỉnh ngưỡng), AO (Analog Output – giá trị điện áp tương tự tỉ lệ với ánh sáng)
- Có biến trở (chiết áp) màu xanh để điều chỉnh độ nhạy
- Có 2 đèn LED báo trạng thái (PWR-LED, DO-LED)
- 4 chân: DO, AO, GND, VCC

c. Cảm biến nhiệt độ, độ ẩm



Hình 2.4 Cảm biến DHT11

Tên: Module cảm biến DHT11
Công dụng: Đo nhiệt độ và độ ẩm không khí, ứng dụng trong trạm thời tiết mini, hệ thống giám sát môi trường
Thông số:

- Điện áp hoạt động: 3.3V – 5.5V DC
- Dải đo độ ẩm: 20–90% RH (sai số ±5%)
- Dải đo nhiệt độ: 0–50°C (sai số ±2°C)
- Chuẩn giao tiếp: 1-wire (tín hiệu số, 1 dây dữ liệu)
- Tần số lấy mẫu: khoảng 1Hz (1 lần/giây)
- 3 chân: VCC (+), OUT (tín hiệu), GND (–)

d. Đèn led



Hình 2.5 Đèn led 5mm

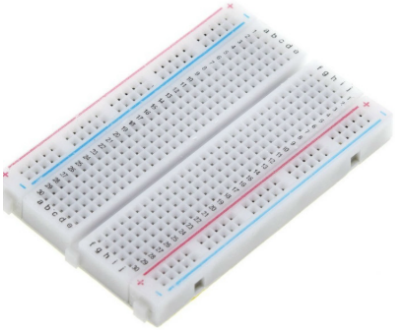
Tên: LED 5mm
Công dụng: Hiển thị trạng thái, tín hiệu báo hiệu trong mạch
Thông số:

- Điện áp thuận: ~1.8 – 2.2V
- Dòng điện: ~20mA
- 2 chân: Anode (dài, +), Cathode (ngắn, –)

e. Một số vật dụng cần thiết khác



Hình 2.6 Dây nối mạch



Hình 2.7 Bảng mạch cắm dây

2.4. Công nghệ sử dụng

a. Công nghệ Backend và cơ sở dữ liệu

Node.js là môi trường thực thi JavaScript phía máy chủ, hỗ trợ xử lý các tác vụ vào/ra bất đồng bộ. Express.js là framework xây dựng ứng dụng web trên Node.js, cung cấp cơ chế định tuyến và xử lý các yêu cầu HTTP. Trong hệ thống IOT CONTROL, Node.js và Express.js được sử dụng để xây dựng Backend và các REST API phục vụ giao tiếp với Frontend. Phần Backend tiếp nhận yêu cầu, xử lý nghiệp vụ, truy xuất dữ liệu và trả kết quả về giao diện. Việc tổ chức mã nguồn theo các lớp Controller và Service giúp phân tách trách nhiệm xử lý, thuận tiện cho việc bảo trì và mở rộng hệ thống.

MySQL là hệ quản trị cơ sở dữ liệu quan hệ, cho phép tổ chức dữ liệu thành các bảng có liên kết với nhau. Trong hệ thống, MySQL được sử dụng để lưu trữ thông tin người dùng, thiết bị, dữ liệu cảm biến và lịch sử điều khiển.

MQTT.js là thư viện cho phép ứng dụng Node.js hoạt động như một MQTT Client. Mosquitto là MQTT Broker trung gian tiếp nhận và phân phối thông điệp giữa các thiết bị kết nối.Trong hệ thống, MQTT.js duy trì kết nối với Mosquitto để tiếp nhận dữ liệu cảm biến do ESP32 gửi lên và chuyển các lệnh điều khiển đến thiết bị thông qua các topic tương ứng. Cơ chế này giúp tách biệt quá trình truyền thông giữa phần cứng và xử lý nghiệp vụ của máy chủ.

WebSocket là công nghệ cho phép thiết lập kết nối hai chiều giữa máy chủ và trình duyệt thông qua một kết nối duy trì liên tục. Thư viện ws hỗ trợ triển khai WebSocket Server trên nền tảng Node.Js.Trong IOT CONTROL, WebSocket được sử dụng để đẩy dữ liệu cảm biến và các thông tin cần cập nhật đến giao diện Dashboard mà không cần liên tục gửi các yêu cầu HTTP mới. Cơ chế Heartbeat sử dụng Ping/Pong có thể được triển khai để kiểm tra tình trạng kết nối và hỗ trợ phát hiện các kết nối không còn hoạt động.

b. Công nghệ Frontend

React là thư viện JavaScript dùng để xây dựng giao diện người dùng theo mô hình thành phần. Vite là công cụ hỗ trợ phát triển và đóng gói ứng dụng Frontend, cung cấp khả năng cập nhật mô-đun nhanh trong quá trình lập trình. Trong đề tài, React được sử dụng để xây dựng các trang Dashboard, dữ liệu cảm biến, lịch sử hoạt động và hồ sơ cá nhân. Vite hỗ trợ khởi chạy môi trường phát triển và đóng gói mã nguồn phục vụ triển khai ứng dụng.

Recharts là thư viện trực quan hóa dữ liệu dành cho React, hỗ trợ xây dựng các dạng biểu đồ thông qua những thành phần có thể tái sử dụng. Trong IOT CONTROL, Recharts được sử dụng để biểu diễn dữ liệu nhiệt độ, độ ẩm và ánh sáng dưới dạng biểu đồ, giúp người dùng theo dõi sự thay đổi của các thông số môi trường theo thời gian. Khi nhận được dữ liệu mới, giao diện có thể cập nhật biểu đồ tương ứng.

c. Công cụ hỗ trợ phát triển và kiểm thử

Visual Studio Code là trình soạn thảo mã nguồn hỗ trợ nhiều ngôn ngữ lập trình và hệ sinh thái tiện ích mở rộng. Công cụ được sử dụng để phát triển mã nguồn Backend, Frontend và các thành phần liên quan của hệ thống.

Postman là công cụ hỗ trợ gửi yêu cầu HTTP và kiểm tra phản hồi từ API. Trong quá trình phát triển, Postman được sử dụng để kiểm thử các phương thức như GET, POST, PUT và DELETE, kiểm tra dữ liệu trả về, mã trạng thái HTTP và khả năng xử lý các trường hợp lỗi.

2.5. Tổng kết chương

Trong chương II, đề tài đã trình bày kiến trúc tổng quan của hệ thống IoT Control, nguyên lý hoạt động của giao thức MQTT và các thành phần phần cứng chính, bao gồm vi điều khiển ESP32, cảm biến ánh sáng, cảm biến nhiệt độ – độ ẩm và đèn LED.

Bên cạnh đó, chương đã giới thiệu các công nghệ được sử dụng để xây dựng hệ thống, bao gồm Node.js và Express.js cho Backend, MySQL để lưu trữ dữ liệu, MQTT phục vụ giao tiếp giữa thiết bị và máy chủ, WebSocket hỗ trợ cập nhật dữ liệu theo thời gian thực, cùng React và Vite để phát triển giao diện người dùng. Các công cụ Visual Studio Code và Postman cũng được sử dụng nhằm hỗ trợ quá trình phát triển và kiểm thử hệ thống.Những nội dung trên là cơ sở để triển khai các chức năng thu thập dữ liệu cảm biến, giám sát môi trường, điều khiển thiết bị và quản lý lịch sử

hoạt động. Đây là cơ sở để thiết kế chi tiết các chức năng, API cho hệ thống ở chương tiếp theo.

Chương III: Thiết kế chi tiết

3.1. Usercase01

Tên Use Case: Xem tổng quan hệ thống và điều khiển thiết bị.
Mã Use Case: UC_DASHBOARD_01
Tác nhân (Actor): Người dùng (User).
Mục đích: Cho phép người dùng theo dõi theo thời gian thực các thông số môi trường (nhiệt độ, độ ẩm, độ sáng), xem biểu đồ lịch sử trong ngày và thao tác bật/tắt nhanh các thiết bị từ xa.
Tiền điều kiện (Pre-conditions):

- Các cảm biến (sensor) và thiết bị (đèn, quạt) đã được kết nối thành công với bộ điều khiển trung tâm và hệ thống mạng.
- Hệ thống backend/server đang hoạt động bình thường để nhận/truyền dữ liệu MQTT

Hậu điều kiện (Post-conditions):

- Trạng thái Bật/Tắt của thiết bị được cập nhật chính xác trên giao diện và thiết bị vật lý thực thi đúng lệnh.
- Hệ thống ghi nhận lại thao tác của người dùng vào log (có thể xem ở màn hình Active History).

Chuỗi sự kiện chính (Main Flow)

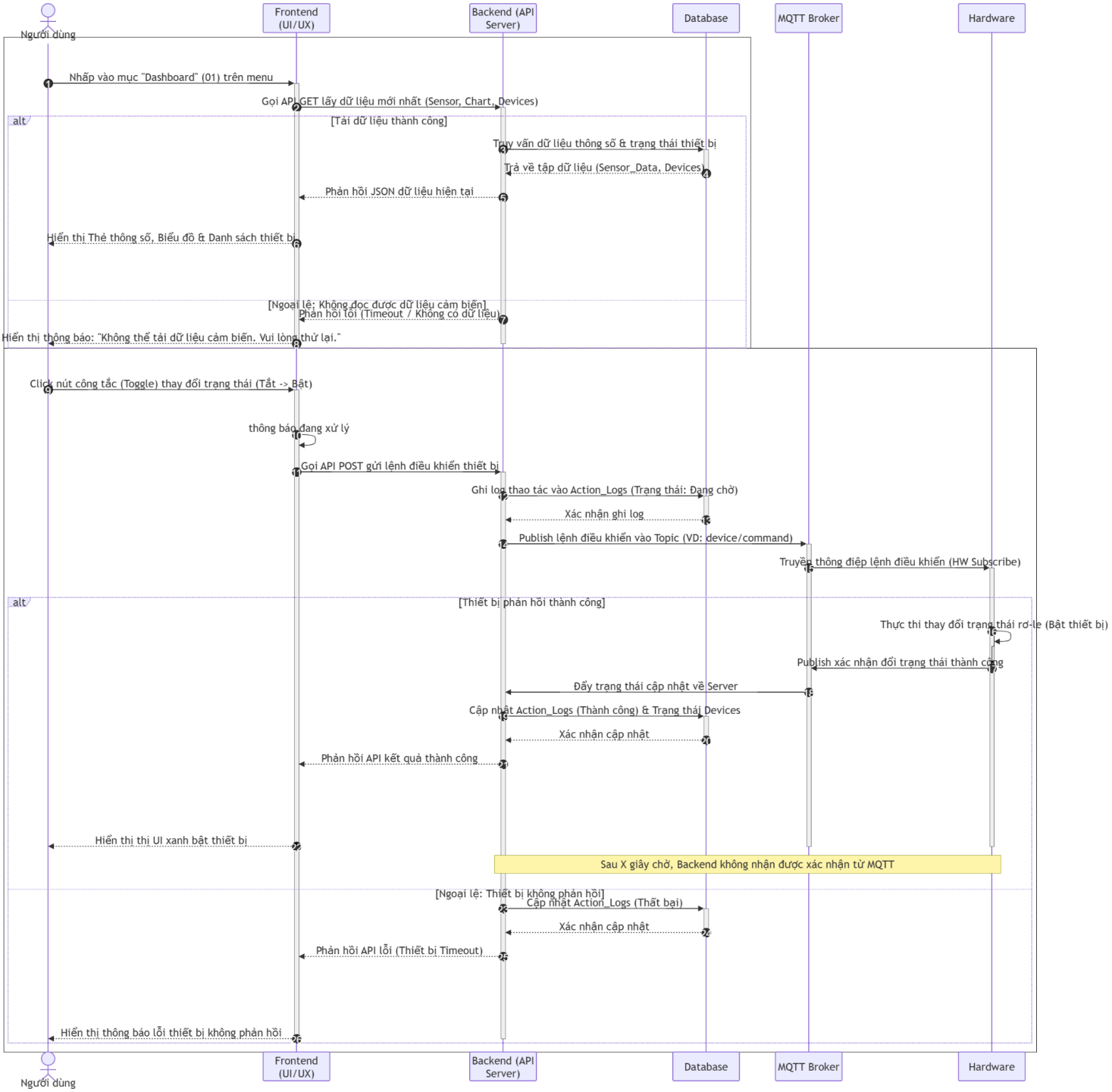
1. Người dùng nhấp vào mục "Dashboard" (01) trên thanh menu điều hướng bên trái.
2. Hệ thống gọi API để lấy dữ liệu mới nhất và hiển thị lên giao diện (dashboard):
    - Thẻ thông số hiện tại: Hiển thị số liệu thực tế của Nhiệt độ (°C), Độ ẩm (%), và Độ sáng (lux).
    - Biểu đồ đường (Line chart): Hiển thị "Biểu đồ giá trị các thông số theo thời gian" với mốc thời gian trong ngày
    - Danh sách Thiết bị (Điều khiển trực tiếp): Hiển thị các thiết bị hiện có (Ví dụ: Đèn 1, Đèn 2, Quạt) cùng trạng thái và vị trí tương ứng (Phòng khách, Phòng ngủ, Ban công).
3. Người dùng click vào nút công tắc (Toggle button) của một thiết bị bất kỳ (ví dụ: Đèn 1) để thay đổi trạng thái từ Tắt -> Bật (hoặc ngược lại).
4. Hệ thống ngay lập tức đổi màu công tắc (ví dụ: chuyển sang màu xanh lá) để báo hiệu đang xử lý lệnh, đồng thời gửi tín hiệu điều khiển xuống thiết bị vật lý thông qua server.
5. Thiết bị vật lý thay đổi trạng thái và gửi tín hiệu xác nhận về hệ thống.
6. Hệ thống cập nhật chữ trạng thái trên UI thành "Bật" (hoặc "Tắt").

Ngoại lệ (Exceptions):

- Hệ thống hiển thị thông báo: "Không thể tải dữ liệu cảm biến. Vui lòng thử lại."

- Hệ thống không đọc được dữ liệu cảm biến thời gian thực
- Thiết bị không phản hồi

Sequence diagram:

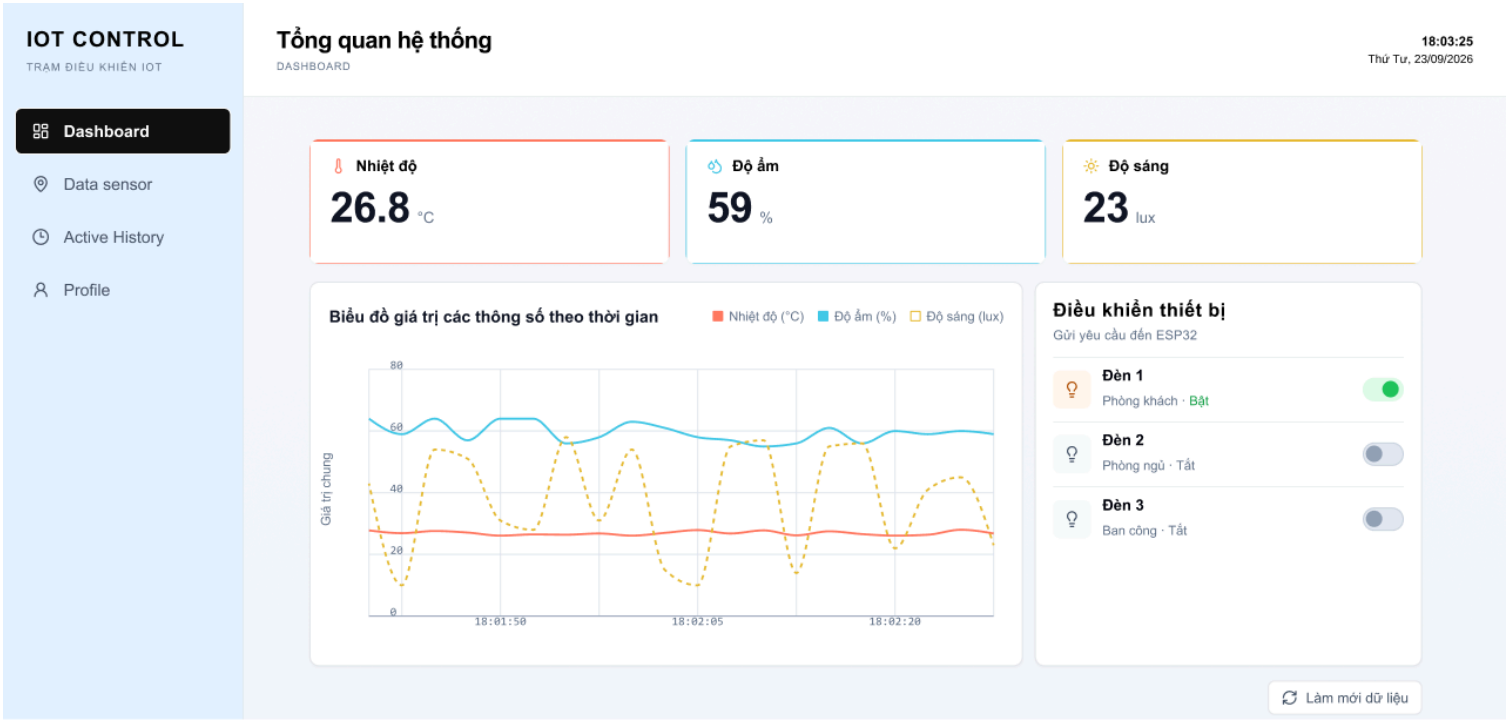


Hình 3.1 sequence diagram dashboard

1. Người dùng nhấp vào mục “Dashboard” (01) trên menu.
2. Frontend gọi API GET để lấy dữ liệu mới nhất gồm Sensor, Chart, Devices từ Backend.

3. Backend truy vấn dữ liệu thông số và trạng thái thiết bị từ Database.
4. Database trả về tập dữ liệu gồm Sensor_Data, Devices cho Backend.
5. Backend phản hồi JSON dữ liệu hiện tại cho Frontend.
6. Frontend xử lý và hiển thị thông số, biểu đồ và danh sách thiết bị cho Người dùng.
7. Trong trường hợp không đọc được dữ liệu cảm biến, Backend phản hồi lỗi Timeout / Không có dữ liệu cho Frontend.
8. Frontend hiển thị thông báo "Không thể tải dữ liệu cảm biến. Vui lòng thử lại." cho Người dùng.
9. Người dùng click nút công tắc (Toggle) để thay đổi trạng thái từ Tắt → Bật.
10. Frontend thông báo đang xử lý.
11. Frontend gọi API POST gửi lệnh điều khiển thiết bị đến Backend.
12. Backend ghi log thao tác vào Action_Logs với trạng thái "Đang chờ" trong Database.
13. Database xác nhận việc ghi log cho Backend.
14. Backend publish lệnh điều khiển vào Topic theo dạng device/command trên MQTT Broker.
15. MQTT Broker truyền thông điệp lệnh điều khiển đến Hardware thông qua HW Subscribe.
16. Hardware thực thi thay đổi trạng thái rơ-le (Bật thiết bị).
17. Hardware publish xác nhận đổi trạng thái thành công lên MQTT Broker.
18. MQTT Broker đẩy trạng thái cập nhật về Server.
19. Backend cập nhật Action_Logs (Thành công) và trạng thái Devices vào Database.
20. Database xác nhận việc cập nhật cho Backend.
21. Backend phản hồi API kết quả thành công cho Frontend.
22. Frontend cập nhật chữ trạng thái trên UI thành "Bật" và hiển thị cho người dùng.
23. Trong trường hợp thiết bị không phản hồi, Backend cập nhật Action_Logs (Thất bại) vào Database.
24. Database xác nhận việc cập nhật cho Backend.
25. Backend phản hồi API lỗi (Thiết bị Timeout) cho Frontend.
26. Frontend khôi phục màu công tắc về trạng thái (Tắt) và hiển thị thông báo lỗi "Thiết bị không phản hồi" cho Người dùng.

Giao diện (Dashboard):



Hình 3.2 Giao diện màn dashboard

Mô tả API:

- Lấy dữ liệu cảm biến mới nhất
  - Method: GET
  - Endpoint: /sensors/latest
  - Query Params: limit (Mặc định 20, tối đa 200).
  - Mô tả: Lấy các bản ghi đo lường mới nhất để vẽ biểu đồ real-time.
  - Response: 200OK

JSON

{
"data": [
{ "timestamp": "2026-10-20T08:15:05.000Z",
"temperature": 29.5, "humidity": 68, "light": 1540 }
],
"latest": { "timestamp": "2026-10-20T08:15:05.000Z",
"temperature": 29.5, "humidity": 68, "light": 1540 }
}

- Lấy danh sách thiết bị
  - Method: GET
  - Endpoint: /devices
  - Mô tả: Lấy danh sách toàn bộ thiết bị (id, tên, vị trí, loại, trạng thái hiện tại).

- Response: Thành công (200 OK):

JSON:


```
{
 "data": [
  {
   "id": "led-1",
   "name": "Đèn phòng khách",
   "location": "Phòng khách",
   "type": "light",
   "status": "OFF",
   "mqttKey": "led_1"
  }
 ]
}
```


- Bật/Tắt thiết bị (Toggle)
  - Method: POST
  - Endpoint: /devices/:id/toggle (Thay :id bằng ID của thiết bị trong bảng devices).
  - Mô tả: Gửi lệnh MQTT để điều khiển bật/tắt thiết bị và ghi log vào bảng action_logs.
  - Body (JSON): { "on": true}
  - Response:
    - 200 OK: ESP32 phản hồi đúng trạng thái trong 3s
    - 404 Not Found: Không tìm thấy thiết bị.
    - 409 Conflict: Thiết bị đang xử lý lệnh trước đó (busy).
    - 504 Gateway Timeout: Thiết bị không phản hồi sau 3 giây.
    - 503 Service Unavailable: Không gửi được lệnh đến MQTT Broker
    - 400 Bad Request: on không phải boolean hoặc thiết bị không có mqttkey

3.2. Usercase02

Tên Use Case: Xem dữ liệu cảm biến (Data Sensor).
Mã Use Case: UC_DATA_SENSOR_02
Tác nhân (Actor): Người dùng (User).
Mục đích: Cho phép người dùng xem danh sách chi tiết các bản ghi số liệu thu thập từ các thiết bị cảm biến, đồng thời hỗ trợ tra cứu chuyên sâu bằng cách lọc theo loại cảm biến, thời gian, và các thông số/ID cụ thể.
Tiền điều kiện (Pre-conditions):

Hệ thống cơ sở dữ liệu đã lưu trữ các bản ghi log gửi về từ cảm biến

Hậu điều kiện (Post-conditions):

- Hiển thị chính xác danh sách các bản ghi dữ liệu tuân theo đúng các tham số truy vấn (query parameters) mà người dùng đã thiết lập.

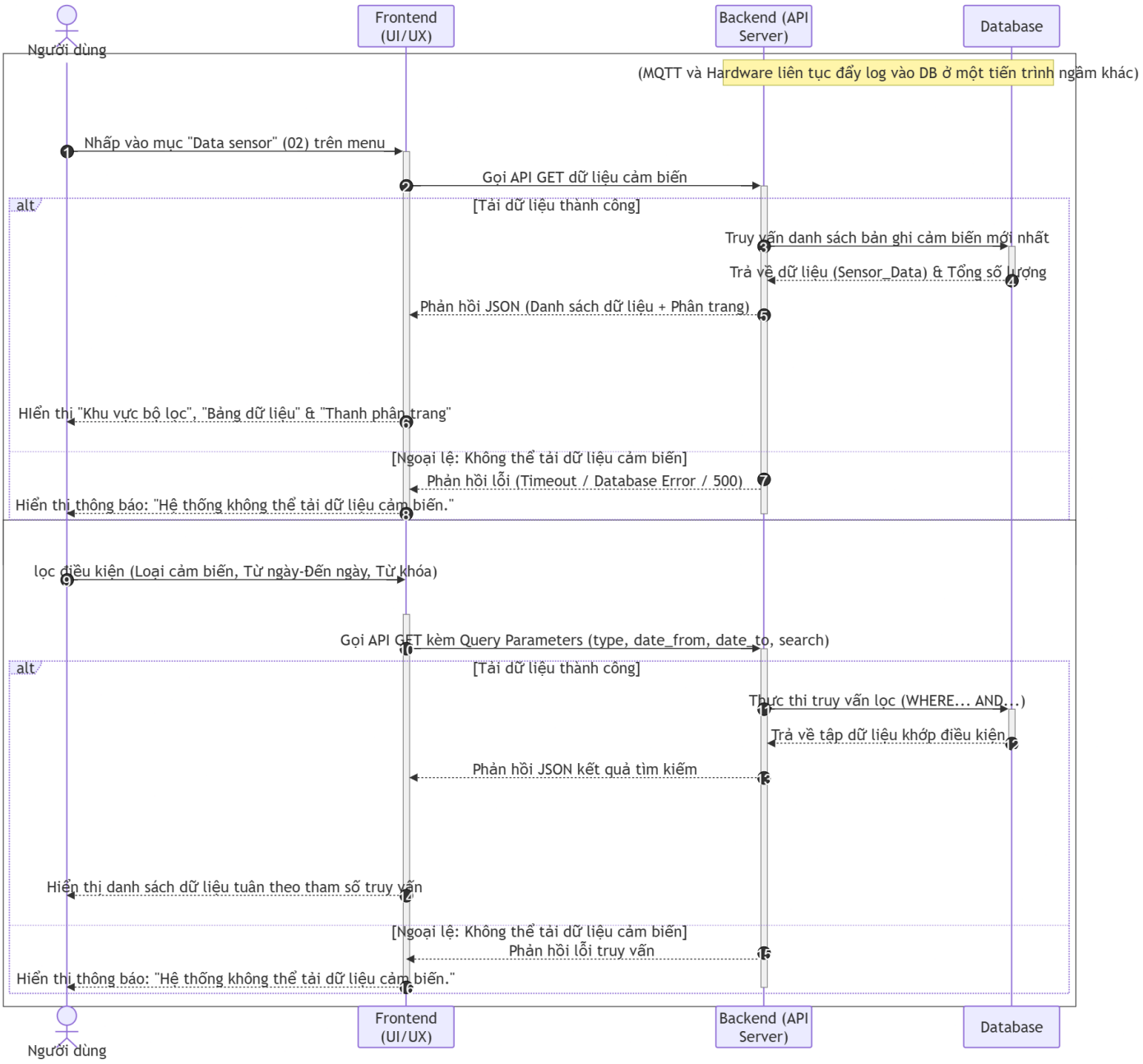
Chuỗi sự kiện chính (Main Flow):

1. Người dùng nhấp vào mục "Data sensor" (02) trên thanh menu điều hướng.
2. Hệ thống gửi request đến API để lấy danh sách dữ liệu cảm biến mới nhất (mặc định được sắp xếp theo thời gian giảm dần)
3. Hệ thống render màn hình "Dữ liệu cảm biến" bao gồm:
    - Khu vực bộ lọc: Dropdown Cách tìm kiếm (mặc định: Tất cả) và ô từ khóa tìm kiếm, Bảng dữ liệu: Chứa các cột ID, LOẠI CẢM BIẾN, GIÁ TRỊ, và THỜI GIAN.
    - Phân trang (Pagination): Hiển thị ở góc dưới cùng (VD: Hiển thị 1-10 / 46 bản ghi, kèm các nút chuyển trang 1, 2, 3, 4, 5) và page size.
4. Người dùng thiết lập các điều kiện tra cứu:
    - Chọn cách tìm kiếm (VD: Chỉ xem Nhiệt độ).
    - Nhập từ khóa vào ô tìm kiếm (VD: ID cảm biến "SN-1042" hoặc toán tử giá trị ">30").
5. Người dùng nhấn nút "Tìm kiếm".
6. Hệ thống xử lý các tham số, gọi lại API và cập nhật state của bảng dữ liệu với kết quả mới trả về.

Ngoại lệ (Exceptions):

- Hệ thống không thể tải dữ liệu cảm biến

Sequence diagram:

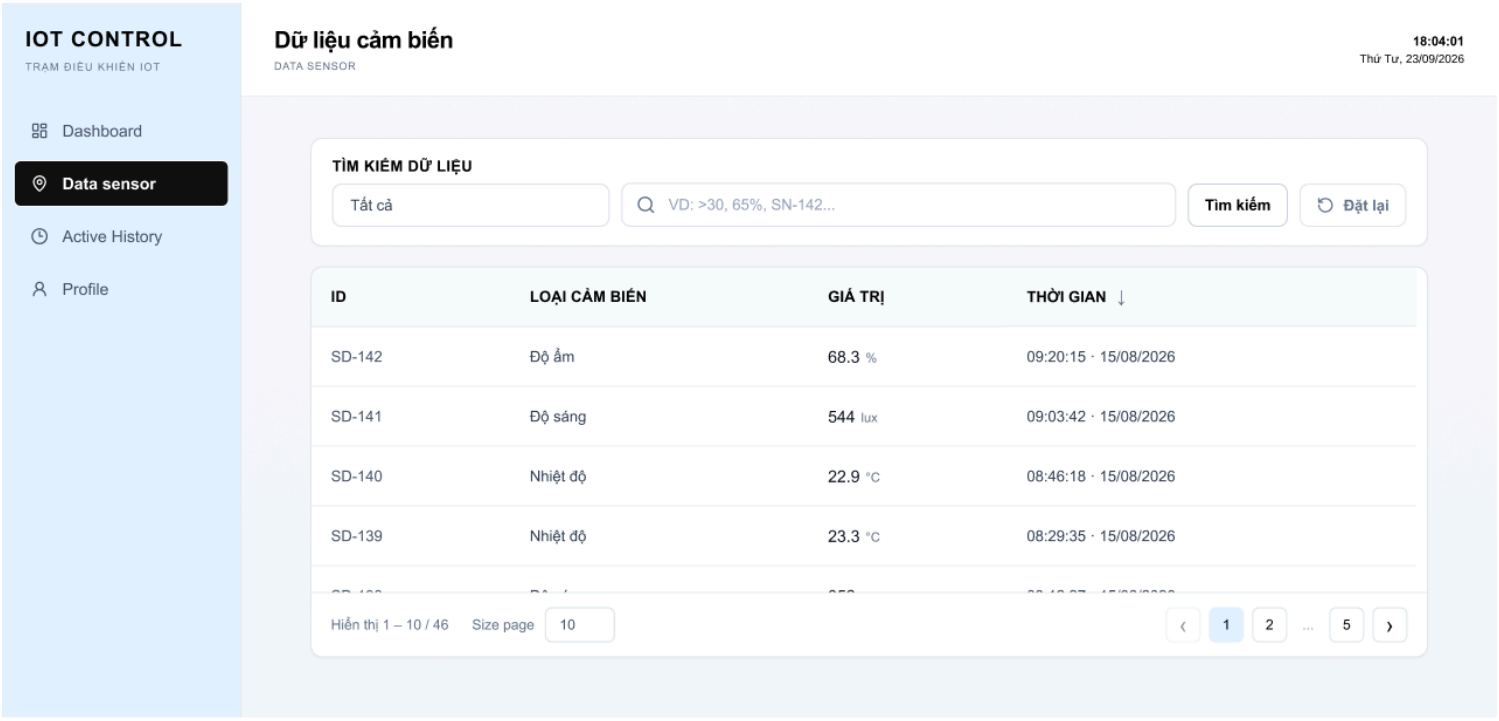


Hình 3.3 Sequence diagram sensor_data

1. Người dùng nhấp vào mục “Data sensor” (02) trên menu.
2. Frontend gọi API GET để lấy dữ liệu cảm biến, mặc định Trang 1 và sắp xếp DESC.
3. Backend truy vấn danh sách các bản ghi cảm biến mới nhất từ Database.
4. Database trả về dữ liệu Sensor_Data và tổng số lượng bản ghi cho Backend.
5. Backend phản hồi JSON, bao gồm danh sách dữ liệu và thông tin phân trang, cho Frontend.
6. Frontend hiển thị dữ liệu “Khu vực bộ lọc”, “Bảng dữ liệu” và “Thanh phân trang” cho Người dùng.

7. Trong trường hợp không thể tải dữ liệu cảm biến, Backend phản hồi lỗi Timeout / Database Error / 500 cho Frontend.
8. Frontend hiển thị thông báo “Hệ thống không thể tải dữ liệu cảm biến.” cho Người dùng.
9. Người dùng nhập hoặc chọn các cách lọc gồm (Loại cảm biến hoặc thời gian),Từ khóa trên Frontend và nhấn nút “Tìm kiếm”.
10. Frontend gọi API GET kèm theo các Query Parameters gồm mode, search, page, pageSizeđến Backend.
11. Backend thực thi truy vấn lọc với các điều kiện WHERE... AND... trên Database.
12. Database trả về tập dữ liệu phù hợp với các điều kiện lọc cho Backend.
13. Backend phản hồi JSON kết quả tìm kiếm cho Frontend.
14. Frontend hiển thị danh sách dữ liệu tuân theo tham số truy vấn cho Người dùng.
15. Trong trường hợp không thể tải dữ liệu cảm biến, Backend phản hồi lỗi truy vấn cho Frontend.
16. Frontend hiển thị thông báo “Hệ thống không thể tải dữ liệu cảm biến.” cho Người dùng.

Giao diện(Datasensor):



Hình 3.4 Giao diện màn sensor_data

Mô tả API:

- Lấy danh sách lịch sử cảm biến (Phân trang)

- Method: GET
- Endpoint: /sensors
- Query Params:
    - mode: all, temperature, humidity, light, time.
    - search: Từ khóa tìm kiếm (có thể dùng toán tử >=, <=, >, <, = cho giá trị float).
    - page: Số trang (mặc định 1).
    - pageSize: Số lượng bản ghi mỗi trang (mặc định 10, tối đa 100).
- Response: Thành công (200 OK)

JSON

```
{
  "data": [ ... ],
  "total": 150,
  "page": 1,
  "pageSize": 10,
  "totalPages": 15
}
```

3.3. Usercase03

Tên Use Case: Tra cứu lịch sử hoạt động thiết bị (Active History).
Mã Use Case: UC_ACTIVE_HISTORY_03
Tác nhân (Actor): Người dùng (User).
Mục đích: Cho phép người dùng theo dõi, kiểm tra lại toàn bộ các sự kiện thay đổi trạng thái (bật, tắt, điều chỉnh) của các thiết bị trong hệ thống, bao gồm nguồn kích hoạt (do người dùng, tự động, hoặc cảm biến) và kết quả thực thi lệnh.
Tiền điều kiện (Pre-conditions):

- Hệ thống backend đã lưu trữ các bản ghi log sự kiện từ các thiết bị và bộ điều khiển trung tâm.

Hậu điều kiện (Post-conditions):

- Hệ thống trả về và hiển thị danh sách các sự kiện hoạt động của thiết bị khớp với tiêu chí tìm kiếm/lọc của người dùng.

Chuỗi sự kiện chính (Main Flow) :

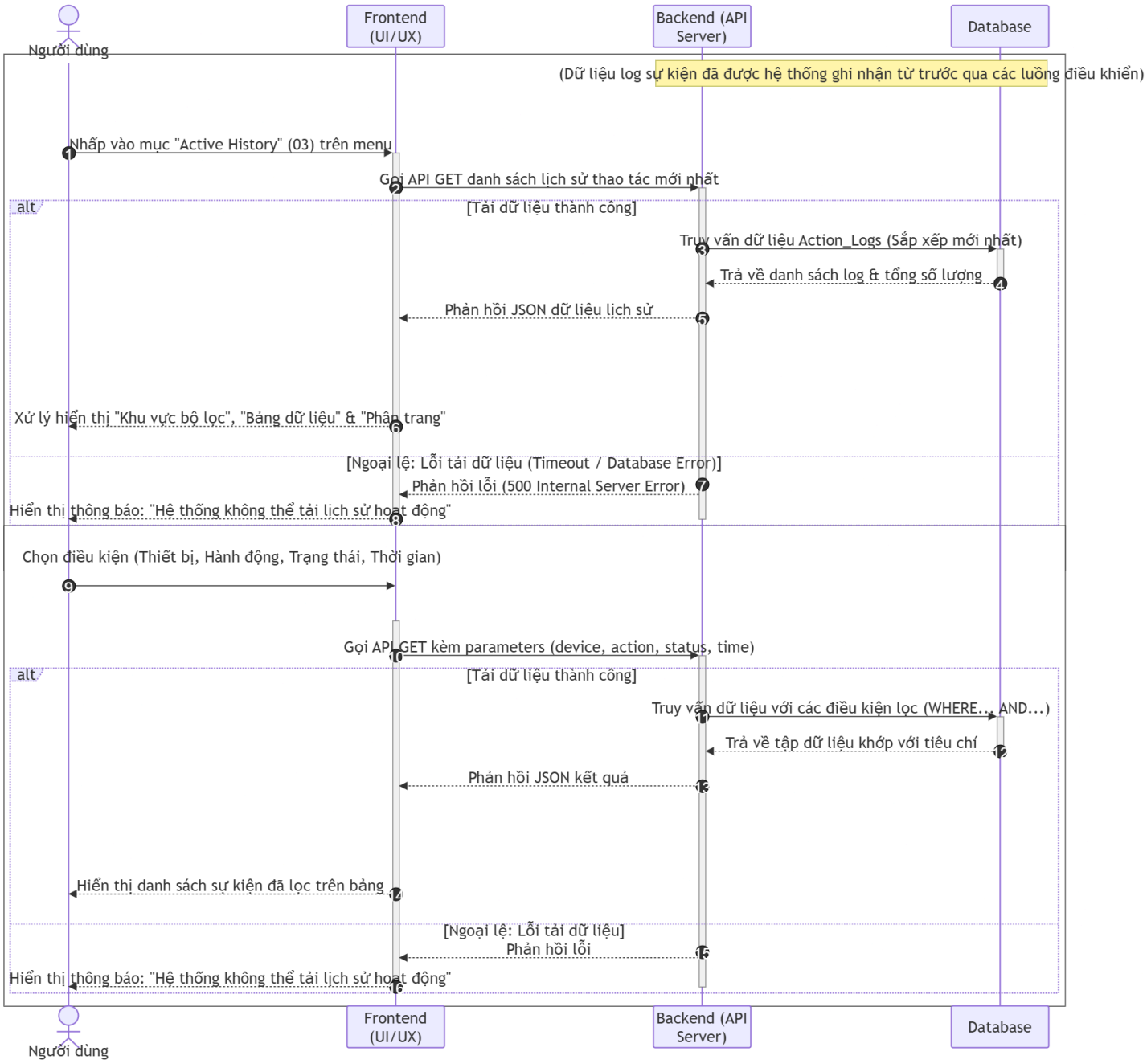
1. Người dùng nhấp vào mục "Active History" (03) trên thanh menu điều hướng.
2. Hệ thống gửi request đến API để lấy danh sách lịch sử thao tác thiết bị mới nhất.
3. Hệ thống render màn hình "Lịch sử hoạt động" bao gồm:
    - Khu vực bộ lọc: Các dropdown Lọc thiết bị, Hành động, Trạng thái và ô nhập thời gian tìm kiếm
    - Bảng dữ liệu: Chứa các cột ID, TÊN THIẾT BỊ (kèm vị trí), HÀNH ĐỘNG, TRẠNG THÁI, THỜI GIAN và NGƯỜI THỰC HIỆN

    - Phân trang (Pagination): Hiển thị số lượng bản ghi (VD: Hiển thị 1-10 / 40), page size và các nút chuyển trang.
4. Người dùng thiết lập các điều kiện tra cứu:
    - Chọn thiết bị cụ thể (VD: Quạt, Điều hòa, Rèm cửa...).
    - Chọn loại hành động (VD: Bật, Tắt, Điều chỉnh).
    - Chọn trạng thái thực thi (VD: Thành công, Thất bại).
    - Nhập thời gian (VD: 14/08/2026).
5. Người dùng nhấn nút "Tìm kiếm".
6. Hệ thống xử lý các tham số, gọi API và cập nhật danh sách hiển thị trên bảng dữ liệu.

Ngoại lệ (Exceptions) :

- Hệ thống không thể tải lịch sử hoạt động

Sequence diagram:

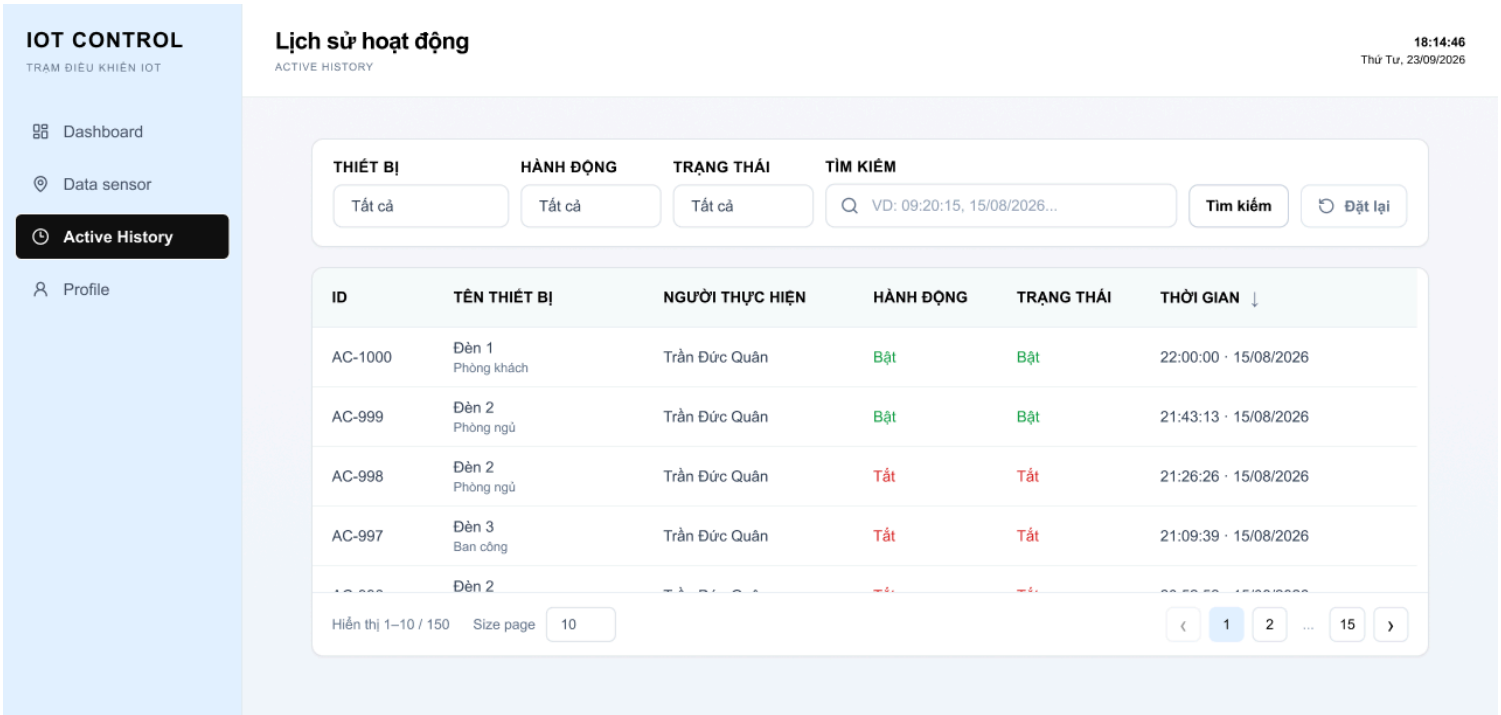


Hình 3.5 Sequence diagram active_history

1. Người dùng nhấp vào mục “Active History” (03) trên menu.
2. Frontend gọi API GET để lấy danh sách lịch sử thao tác mới nhất.
3. Backend truy vấn dữ liệu Action_Logs, sắp xếp theo thời gian mới nhất, từ Database.
4. Database trả về danh sách log và tổng số lượng cho Backend.
5. Backend phản hồi JSON dữ liệu lịch sử cho Frontend.
6. Frontend hiển thị “Khu vực bộ lọc”, “Bảng dữ liệu” và “Phân trang” cho Người dùng.

7. Trong trường hợp lỗi tải dữ liệu (Timeout / Database Error), Backend phản hồi lỗi 500 Internal Server Error cho Frontend.
8. Frontend hiển thị thông báo “Hệ thống không thể tải lịch sử hoạt động” cho Người dùng.
9. Người dùng chọn các điều kiện lọc gồm Thiết bị, Hành động, Trạng thái và ô nhập Thời gian và nhấn nút “Tìm kiếm”.
10. Frontend gọi API GET kèm các parameters device, action, status, time đến Backend.
11. Backend truy vấn dữ liệu với các điều kiện lọc WHERE... AND... trên Database.
12. Database trả về tập dữ liệu khớp với tiêu chí lọc cho Backend.
13. Backend phản hồi JSON kết quả cho Frontend.
14. Frontend hiển thị danh sách sự kiện đã lọc trên bảng cho Người dùng.
15. Trong trường hợp lỗi tải dữ liệu, Backend phản hồi lỗi cho Frontend.
16. Frontend hiển thị thông báo “Hệ thống không thể tải lịch sử hoạt động” cho Người dùng.

Giao diện (Activehistory):



Hình 3.6 Giao diện màn active_history

Mô tả API:

- Lấy danh sách lịch sử hành động (Phân trang)
  - Method: GET
  - Endpoint: /actions

- Query Params:
    - device: ID thiết bị hoặc all.
    - action: all, Bật, Tắt.
    - status: all, SUCCESS, FAILED.
    - search: Từ khóa tìm kiếm (lọc theo thời gian).
    - sort: asc hoặc desc.
    - page: Số trang (mặc định 1).
    - pageSize: Số lượng bản ghi mỗi trang (mặc định 10, tối đa 100).
- Mô tả: Trả về danh sách thao tác do ai thực hiện, vào thời gian nào, thiết bị gì và trạng thái lệnh.
- Response: Thành công (200 OK)

JSON

```
{
  "data": [ ... ],
  "total": 150,
  "page": 1,
  "pageSize": 10,
  "totalPages": 15
}
```

3.4. Usercase04

Tên Use Case: Xem và sửa hồ sơ cá nhân (Profile).
Mã Use Case: UC_PROFILE_04
Tác nhân (Actor): Người dùng (User).
Mục đích: Cho phép người dùng xem lại thông tin định danh cá nhân của tài khoản đang đăng nhập, cũng như truy cập nhanh vào các tài nguyên/liên kết liên quan đến dự án (Github, Figma, File báo cáo).
Tiền điều kiện (Pre-conditions):

- Hệ thống backend đã lưu trữ thông tin tài khoản của người dùng (từ database hoặc hệ thống SSO nếu có).

Hậu điều kiện (Post-conditions):

- Hệ thống truy xuất và hiển thị chính xác các thông tin cá nhân lên giao diện.

Chuỗi sự kiện chính (Main Flow):

1. Người dùng nhấp vào mục "Profile" (04) trên thanh menu điều hướng bên trái.
2. Hệ thống gửi request đến API (ví dụ: /api/profile) để lấy thông tin chi tiết của tài khoản hiện tại.
3. Hệ thống render màn hình "Hồ sơ cá nhân" và hiển thị các trường dữ liệu bên trong một khối trung tâm (Card), bao gồm:
    - Hình ảnh đại diện (Avatar): Hiển thị ảnh profile hoặc chữ viết tắt.
    - Thông tin định danh cơ bản: TÊN, MÃ SINH VIÊN.

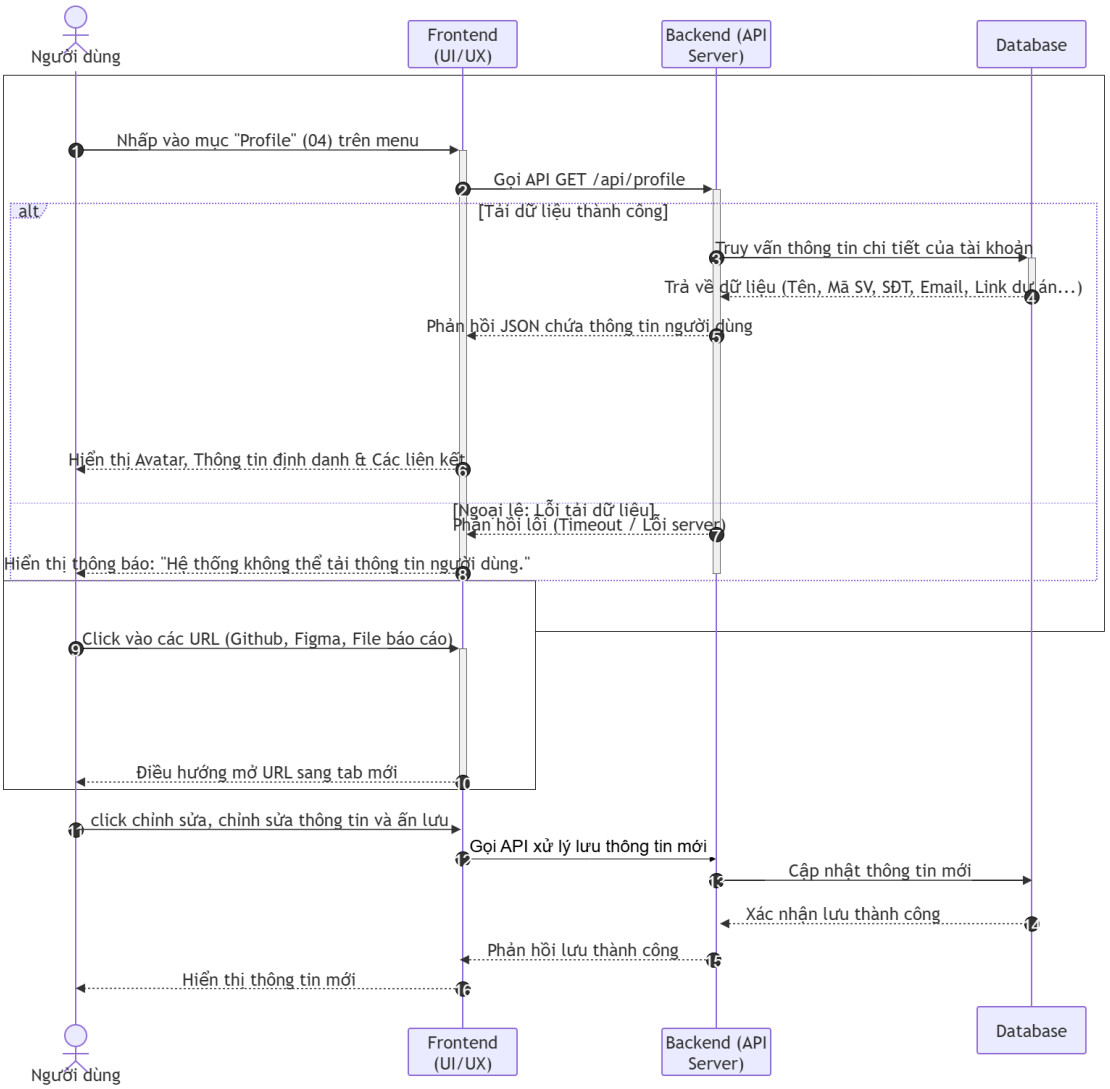
- Thông tin liên hệ: EMAIL, SỐ ĐIỆN THOẠI.
- Liên kết tài nguyên dự án: LINK GITHUB, LINK FIGMA, LINK FILE BÁO CÁO.

4. Người dùng xem thông tin trên màn hình.
5. (Tùy chọn) Người dùng click vào các liên kết để truy cập vào mã nguồn, bản thiết kế hoặc tài liệu báo cáo.
6. Hệ thống mở các liên kết này sang một tab trình duyệt mới (target="_blank").
7. Người dùng chỉnh sửa thông ti.
8. Hệ thống lưu thông tin mới.

Ngoại lệ (Exceptions):

- Hệ thống không thể tải thông tin người dùng.

Sequence diagram:

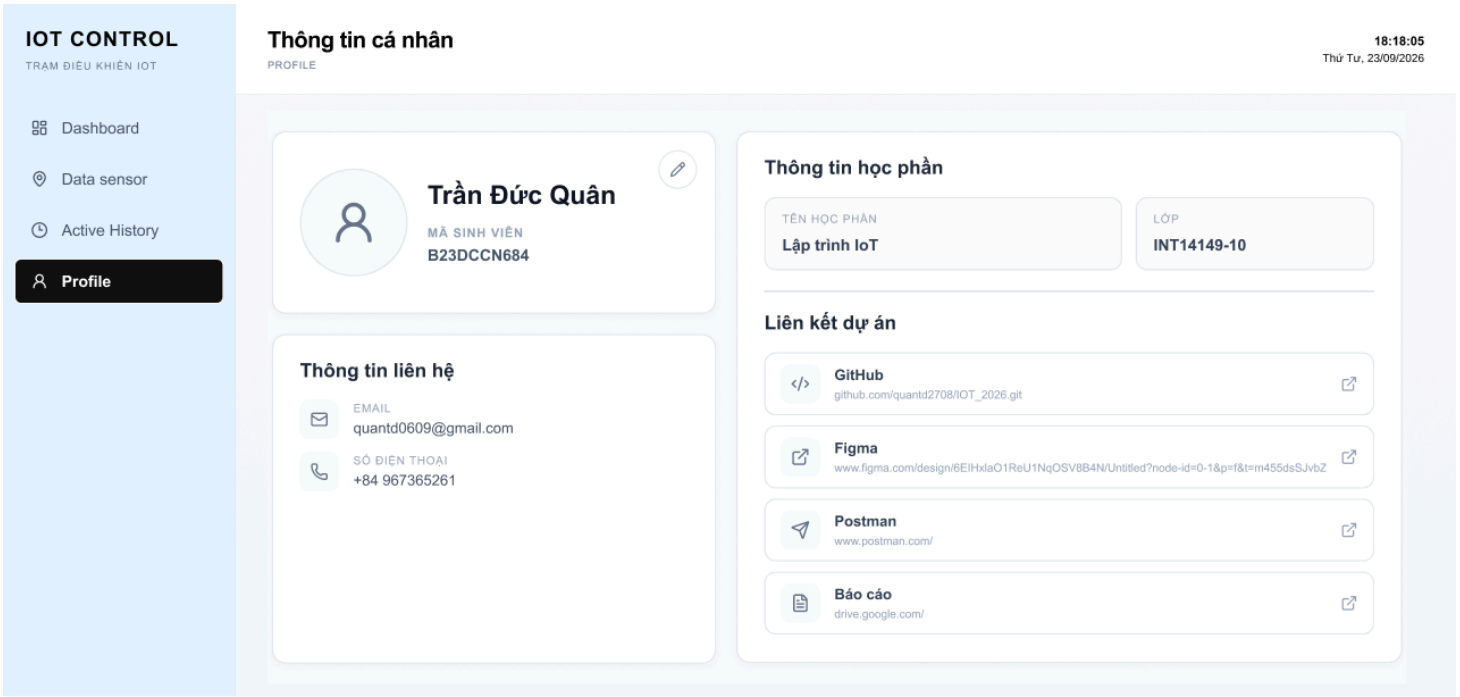


Hình 3.7 Sequence diagram profile

1. Người dùng nhấp vào mục “Profile” (04) trên menu.

2. Frontend gọi API GET /api/profile đến Backend.
3. Backend truy vấn thông tin chi tiết của tài khoản từ Database.
4. Database trả về dữ liệu gồm Tên, Mã SV, SĐT, Email, Link dự án,... cho Backend.
5. Backend phản hồi JSON chứa thông tin người dùng cho Frontend.
6. Frontend hiển thị Avatar, thông tin định danh và các liên kết cho Người dùng.
7. Trong trường hợp lỗi tải dữ liệu, Backend phản hồi lỗi Timeout / Lỗi server cho Frontend.
8. Frontend hiển thị thông báo "Hệ thống không thể tải thông tin người dùng." cho Người dùng.
9. Người dùng click vào các URL như Github, Figma, File báo cáo.
10. Frontend điều hướng mở URL sang Tab mới
11. Người dùng click vào chỉnh sửa, chỉnh sửa rồi ấn lưu.
12. Frontend gọi API xử lý lưu thông tin mới
13. Backend cập nhật thông tin mới vào database
14. Database xác nhận lưu thành công
15. Backend phản hồi lưu thành công
16. Frontend hiển thị thông tin mới cho người dùng

Giao diện (Profile):



Hình 3.8 Giao diện màn profile

Mô tả API:

- Lấy thông tin cá nhân
    - Method: GET

- Endpoint: /profile
- Mô tả: Lấy thông tin hồ sơ của người dùng hiện tại.
- Response: Thành công (200 OK):

JSON


```
{
   "name": "",
   "studentId": "B23DCCN684",
   "email": "quantd0609@gmail.com",
   "phone": "+84 967365261",
   "course": "Lập trình IoT",
   "className": "INT14149-10",
   "github": "https://github.com/quantd2708/IOT_2026.git",
  "figma": "https://www.figma.com/",
  "postman": "https://quantd0609-5144059.postman.co/",
   "cv": "https://drive.google.com/",
 "avatar": "http://localhost:5000/uploads/avatars/avatar"
}
```


- Cập nhật thông tin cá nhân
  - Method: PUT
  - Endpoint: /profile
  - Mô tả: Cập nhật thông tin hồ sơ người dùng.
  - Body (JSON): Chú ý độ dài tối đa dựa theo thiết kế bảng users.
    - name (Bắt buộc, max 100 ký tự)
    - studentId (Bắt buộc, max 20 ký tự)
    - email (Bắt buộc, max 100 ký tự, định dạng email chuẩn)
    - phone (Tùy chọn, max 20 ký tự)
    - course (Tùy chọn, max 100 ký tự)
    - className (Tùy chọn, max 50 ký tự)
    - github, figma, postman, cv (Tùy chọn, max 255 ký tự, phải là chuẩn URL http/https)
  - Response:
    - 200 OK: Hồ sơ sau cập nhật
    - 400 Bad Request: Thiếu trường bắt buộc hoặc sai định dạng.
    - 404 Not Found: không tìm thấy người dùng.
    - 409 Conflict : email hoặc mã sinh viên đã tồn tại.
- Upload ảnh đại diện
  - Method: POST
  - Endpoint: /profile/avatar
  - Body (form-data):
    - avatar: File ảnh (Chỉ nhận .jpg, .png, .webp, .gif. Tối đa 2MB).

        - Response
            - 200 OK: Trả về object thông tin người dùng có chứa đường dẫn avatar mới.
            - 400 Bad Request : chưa chọn ảnh, sai định dạng, hoặc quá 2MB
            - 404 Not Found: không tìm thấy người dùng.
- Xóa ảnh đại diện
    - Method: DELETE
    - Endpoint: /profile/avatar
    - Mô tả: Xóa ảnh đại diện hiện tại và set trường avatar_url trong bảng users về NULL.
    - Response:
        - 200 OK: hồ sơ với avatar rỗng.
        - 404 Not Found: không tìm thấy người dùng.

3.5. Usercase05

Tên Use Case: Tự động thu thập, hiện thị và lưu trữ dữ liệu cảm biến.
Mã Use Case: UC_SYSTEM_SENSOR_LOG_05
Tác nhân (Actor): Thiết bị cảm biến (Sensor) / Hệ thống (System).
Mục đích: Đảm bảo dữ liệu môi trường (nhiệt độ, độ ẩm, độ sáng) được ghi nhận liên tục theo chu kỳ cài đặt sẵn để phục vụ cho việc vẽ biểu đồ và tra cứu lịch sử.
Tiền điều kiện (Pre-conditions):

- Thiết bị cảm biến đã được cấp nguồn, kết nối mạng và hoạt động bình thường.
- Hệ thống Backend và MQTT Broker đang lắng nghe (subscribe) các luồng dữ liệu.

Hậu điều kiện (Post-conditions):

- Dữ liệu mới được insert thành công vào bảng Sensor_Data trong cơ sở dữ liệu kèm theo mốc thời gian (timestamp) chính xác.

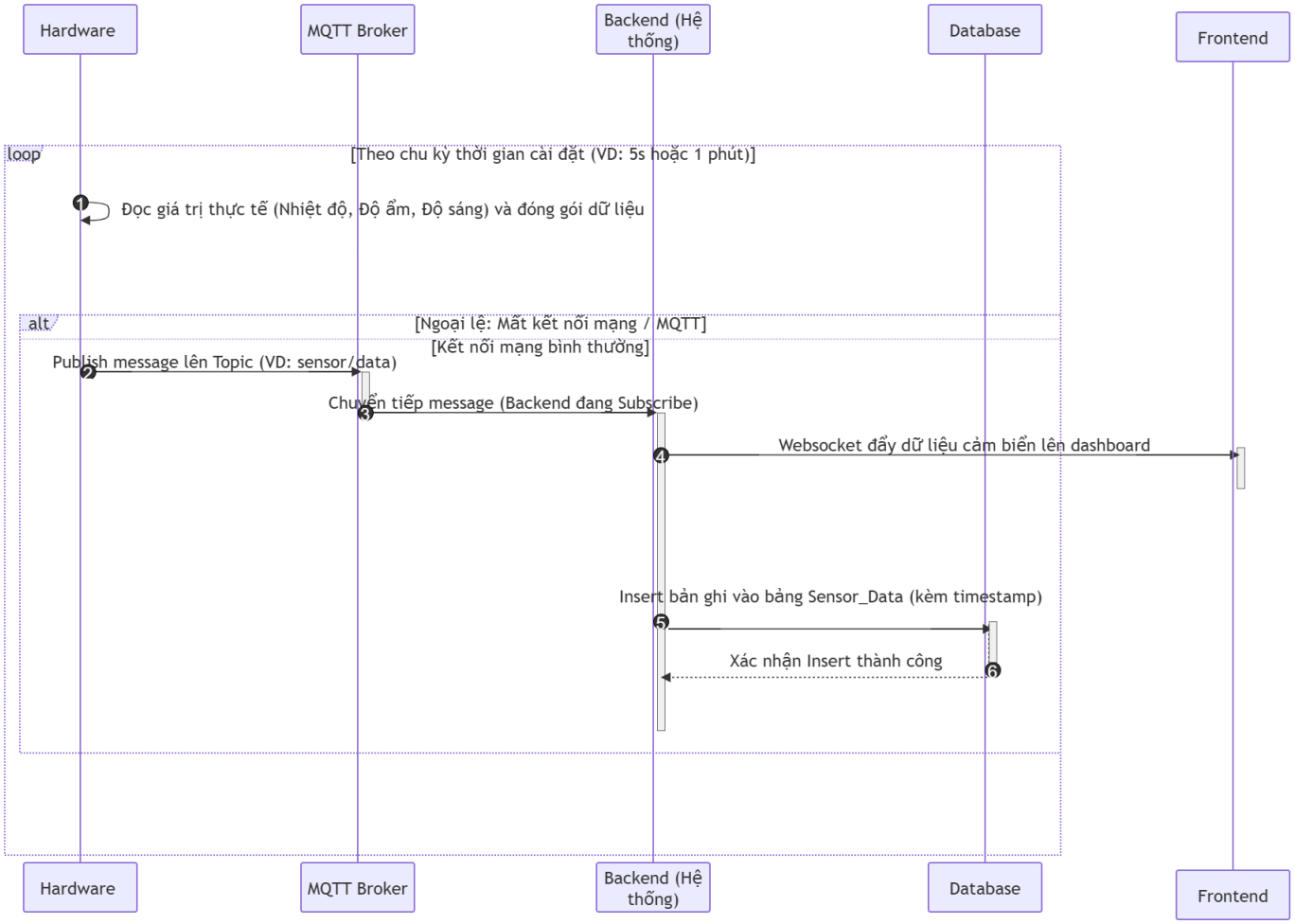
Chuỗi sự kiện chính (Main Flow):

1. Cảm biến đọc giá trị môi trường thực tế (Nhiệt độ, độ ẩm, độ sáng) theo chu kỳ thời gian đã cài đặt (Ví dụ: mỗi 5 giây hoặc 1 phút).
2. Cảm biến đóng gói dữ liệu và gửi thông điệp (Publish message) lên MQTT Broker.
3. MQTT Broker tiếp nhận và chuyển tiếp dữ liệu đến hệ thống Backend (đang Subscribe).
4. Backend nhận dữ liệu và đẩy dữ liệu lên dashboard qua Websocket.
5. Backend tạo bản ghi mới và lưu vào cơ sở dữ liệu (Database).

Ngoại lệ (Exceptions):

- Mất kết nối mạng/MQTT: Cảm biến không thể gửi dữ liệu lên Broker.

Sequence diagram:



Hình 3.9 Sequence diagram chức năng tự ghi dữ liệu cảm biến

1. Hardware đọc các giá trị thực tế gồm Nhiệt độ, Độ ẩm và Độ sáng theo chu kỳ thời gian đã cài đặt, ví dụ 5 giây hoặc 1 phút và đóng gói dữ liệu (Payload).
2. Hardware publish message lên MQTT Topic.
3. MQTT Broker chuyển tiếp message đến Backend đang Subscribe Topic tương ứng.
4. Backend đẩy dữ liệu realtime lên cho dashboard bằng Websocket
5. Backend thực hiện Insert bản ghi vào bảng Sensor_Data, kèm theo timestamp, trong Database.
6. Database gửi xác nhận Insert thành công về Backend.

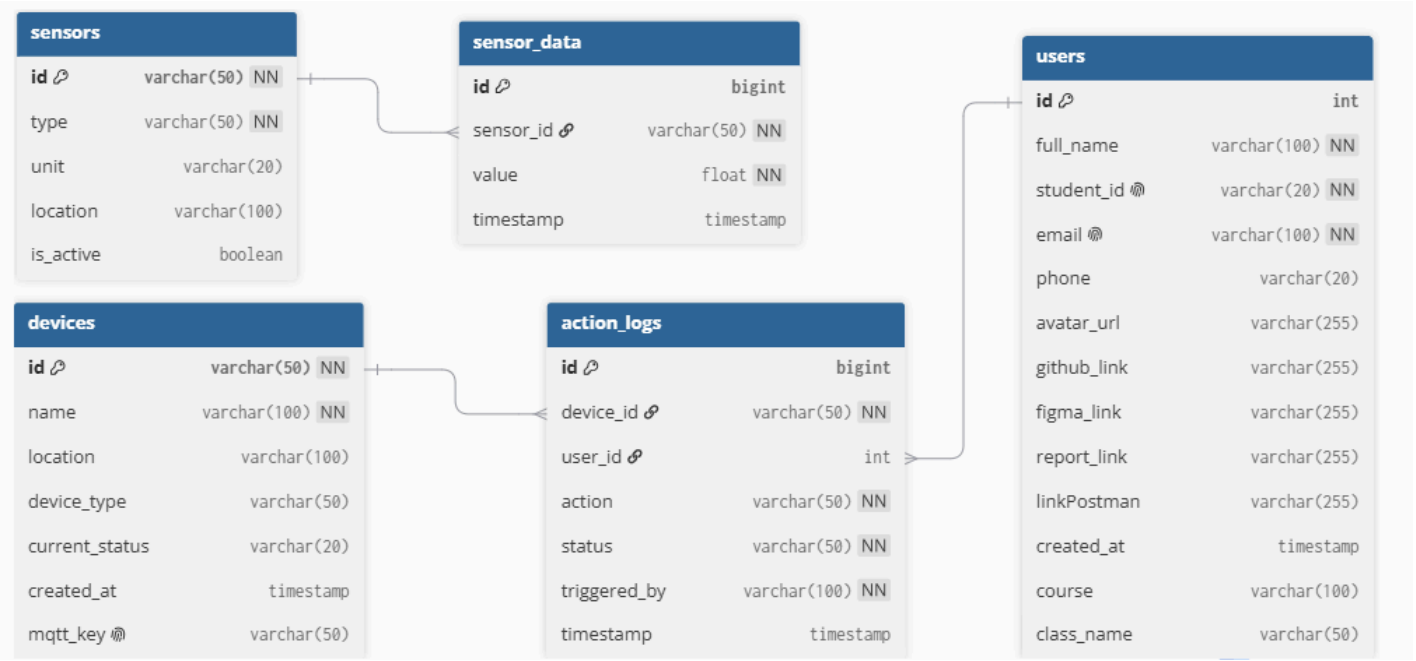
Mô tả Websocket: WS /ws

Kết nối tại ws://<host>:<port>/ws, dùng chung cổng với REST API. Kết nối không cần tham số. Client chỉ lắng nghe, không gửi thông điệp nào lên server.(senson: mỗi khi nhận được một lần đo hợp lệ từ thiết bị (khoảng 5 giây/lần), device: khi thiết bị báo trạng thái bật/tắt mới).

Thông điệp JSON:

{ "type": "sensor", "data": { "timestamp": "2026-10-20T08:15:05.000Z", "temperature": 29.5, "humidity": 68, "light": 1540 } }
{ "type": "device", "data": { "id": 1, "on": true } }

3.6. ERD



Hình 3.10 Sơ đồ ERD

3.7. Danh sách topic

STT	Topic	Payload	Mục đích
1	b23dccn684/esp32/sensor/data	JSON: {"temp":29.5,"hum":68,"light":1540}	Gửi gói dữ liệu tổng hợp (nhiệt độ, độ ẩm, độ sáng) dưới định dạng JSON từ phần cứng (ESP32) lên Server theo chu kỳ 5 giây một lần
2	b23dccn684/esp32/device/command/{mqtt_key}	ON hoặc OFF	Server gửi lệnh Bật/Tắt (chuỗi "ON" hoặc "OFF") xuống thiết bị phần cứng cụ thể. {mqtt_key} tương ứng với led1, led2, led3.
3	b23dccn684/esp32/device/status/{mqtt_key}	ON hoặc OFF	Phần cứng phản hồi trạng thái thực tế (chuỗi "ON" hoặc "OFF") lên Server ngay sau khi thực thi lệnh. Dùng để Server giải phóng Timeout chờ (để UI báo thành công), cập nhật trạng thái vào DB và ghi nhật ký Action Logs.

Bảng 3.1 Danh sách topic

3.8. Tổng kết chương

Trong chương III, đề tài đã trình bày thiết kế chi tiết các chức năng chính của hệ thống IoT Control, bao gồm xem tổng quan và điều khiển thiết bị, theo dõi dữ liệu cảm biến, tra cứu lịch sử hoạt động, quản lý hồ sơ cá nhân và tự động thu thập, lưu trữ dữ liệu cảm biến. Với mỗi chức năng, chương đã mô tả các tác nhân, điều kiện thực hiện, luồng xử lý chính, các trường hợp ngoại lệ và sơ đồ tuần tự nhằm làm rõ cách các thành phần trong hệ thống tương tác với nhau.

Bên cạnh đó, chương đã xác định các API phục vụ việc trao đổi dữ liệu giữa Frontend và Backend, đồng thời trình bày cấu trúc cơ sở dữ liệu thông qua sơ đồ ERD và danh sách các MQTT Topic dùng để truyền dữ liệu cảm biến, gửi lệnh điều khiển và nhận trạng thái phản hồi từ thiết bị ESP32.

Những nội dung thiết kế trên là cơ sở để triển khai các chức năng của hệ thống, bảo đảm sự phối hợp giữa giao diện người dùng, máy chủ, cơ sở dữ liệu và thiết bị

phần cứng. Đây cũng là nền tảng cho quá trình xây dựng, tích hợp và kiểm thử hệ thống trong giai đoạn tiếp theo.

Chương IV: Triển khai hệ thống và kết quả

4.1. Lắp ráp phần cứng

Các chân cảm biến và đèn led được nối với mạch điều khiển trung tâm(Board ESP32 DevKit) theo bảng sau:

STT	Thiết bị	Chân trên Module	Chân trên ESP32
1	Cảm biến ánh sáng	VCC	3V3
		GND	GND
		AO (Analog Out)	D34
2	Cảm biến DHT11	+(VCC)	3V3
		-(GND)	GND
		Out	D4
3	Đèn LED 1 (Đỏ)	Chân dài (Anode)	D13
		Chân ngắn (Cathode)	GND
4	Đèn LED 2 (Vàng)	Chân dài (Anode)	D12
		Chân ngắn (Cathode)	GND
5	Đèn LED 3 (Lục)	Chân dài (Anode)	D14
		Chân ngắn (Cathode)	GND

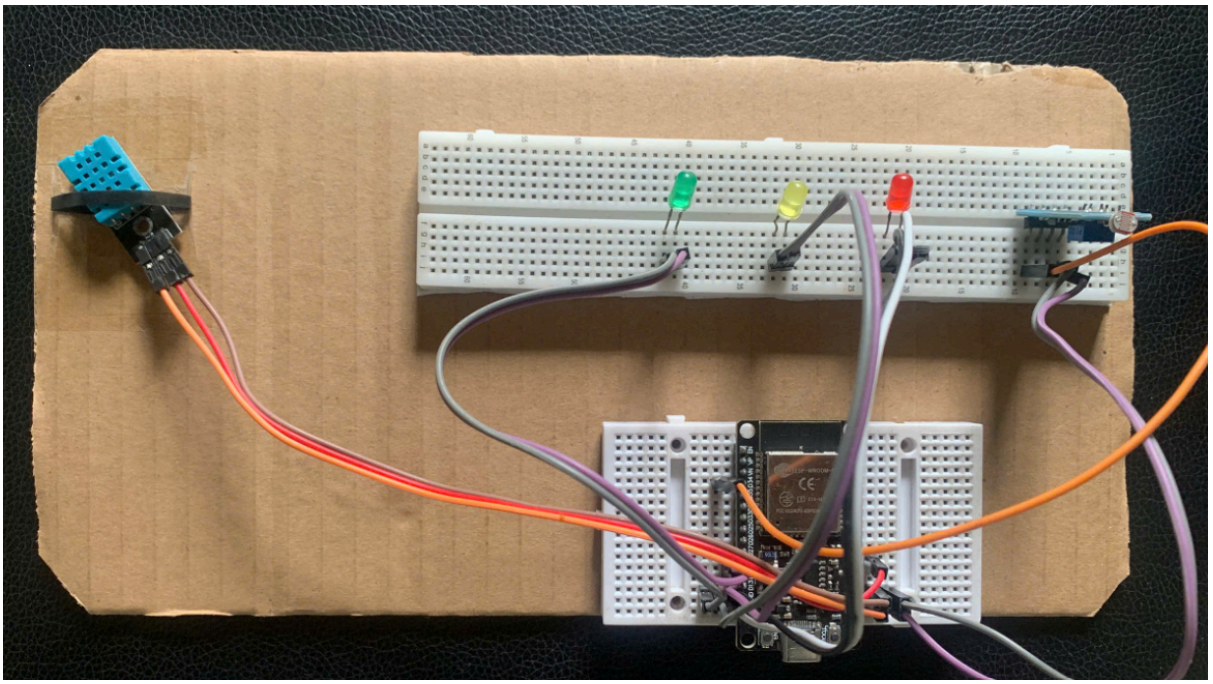
Bảng 4.1 Kết nối các chân mạch

Trong mô hình, ESP32 DevKit đóng vai trò là bộ điều khiển trung tâm, chịu trách nhiệm thu thập dữ liệu từ các cảm biến và điều khiển trạng thái của các đèn LED. Các chân kết nối được lựa chọn dựa trên chức năng của từng thiết bị, cụ thể như sau:

- Chân 3V3 và GND: Cung cấp nguồn điện 3,3 V và đường nối đất chung cho cảm biến ánh sáng, cảm biến DHT11 và các đèn LED, giúp các thiết bị hoạt động ổn định và bảo đảm tín hiệu điện có cùng mức tham chiếu.

- Chân D34 (GPIO34): Được sử dụng để tiếp nhận tín hiệu analog AO từ cảm biến ánh sáng. ESP32 sử dụng bộ chuyển đổi tương tự – số (ADC) để đọc giá trị điện áp đầu ra, từ đó xác định mức độ sáng của môi trường. GPIO34 là chân chỉ có chức năng đầu vào, phù hợp với nhiệm vụ thu thập dữ liệu cảm biến.
- Chân D4 (GPIO4): Được sử dụng để nhận dữ liệu từ chân OUT của cảm biến nhiệt độ và độ ẩm DHT11. Thông qua giao thức truyền dữ liệu riêng của cảm biến và thư viện DHT.h, ESP32 đọc được các thông số nhiệt độ và độ ẩm môi trường.
- Chân D13 (GPIO13), D12 (GPIO12) và D14 (GPIO14): Được cấu hình làm các chân đầu ra số để điều khiển lần lượt đèn LED đỏ, vàng và lục. Khi chân GPIO được thiết lập ở mức HIGH, đèn LED tương ứng sáng; khi được thiết lập ở mức LOW, đèn LED tắt. Các đèn được sử dụng để minh họa trạng thái điều khiển thiết bị từ hệ thống.

Việc phân chia các chân kết nối theo chức năng đầu vào và đầu ra giúp ESP32 thực hiện đồng thời nhiệm vụ giám sát môi trường và điều khiển thiết bị. Sau khi hoàn thành lắp ráp, các kết nối được kiểm tra trước khi triển khai chương trình nhúng và thiết lập giao tiếp MQTT với máy chủ.



Hình 4.1 Mô hình thực tế sau khi lắp ráp

4.2. Triển khai phần mềm nhúng trên ESP32

Phần mềm nhúng cho ESP32 được phát triển trên nền tảng Arduino IDE bằng ngôn ngữ C/C++. Quá trình lập trình được chia thành ba luồng xử lý chính để đảm bảo tính thời gian thực.

a. Lập trình thu thập dữ liệu cảm biến

ESP32 sử dụng thư viện DHT.h để giao tiếp với cảm biến DHT11 qua chân D4 và hàm analogRead() để đọc dữ liệu thô từ quang trở qua chân D34. Nhằm tránh hiện tượng nghẽn (blocking) toàn bộ hệ thống, thuật toán sử dụng hàm millis() để định tuyến chu kỳ đọc dữ liệu mỗi 5 giây (5000ms) một lần thay vì dùng hàm delay(). Dữ liệu đo được (nhiệt độ, độ ẩm, độ sáng) được đóng gói thành chuỗi định dạng JSON {"temp":...,"hum":...,"light":...} để chuẩn bị gửi lên máy chủ.

b. Kết nối Wi-Fi và MQTT

Vi điều khiển sử dụng thư viện WiFi.h để kết nối vào mạng cục bộ và thư viện PubSubClient.h để giao tiếp với MQTT Broker (Mosquitto) tại địa chỉ IP 172.20.10.4 thông qua cổng 1884 và yêu cầu xác thực bằng tập tin mật khẩu với user: admin password: 123456

- Sau khi thiết lập kết nối thành công, ESP32 sẽ tự động xuất bản (publish) chuỗi JSON chứa dữ liệu cảm biến lên topic .../sensor/data theo chu kỳ 5 giây.
- Hệ thống tích hợp sẵn hàm reconnect(), giúp ESP32 tự động thử kết nối lại sau mỗi 5 giây nếu phát hiện rớt mạng Wi-Fi hoặc mất kết nối với MQTT Broker.

c. Xử lý lệnh điều khiển

Trong vòng lặp chính client.loop(), ESP32 duy trì lắng nghe (subscribe) các topic nhận lệnh điều khiển như .../command/led1, led2, led3 và lệnh tổng all. Khi hàm callback bắt được tín hiệu mang nội dung "ON" hoặc "OFF":

1. Hệ thống chuyển đổi trạng thái tín hiệu tại các chân GPIO (D13, D12, D14) tương ứng bằng hàm digitalWrite(LED_PIN, HIGH/LOW) để bật/tắt thiết bị vật lý.
2. Lập tức xuất bản (publish) thông điệp phản hồi "ON" hoặc "OFF" lên topic trạng thái (.../status/ledX) để báo cáo kết quả thực thi về lại máy chủ trung tâm.

4.3. Triển khai Backend và cơ sở dữ liệu

Hệ thống máy chủ được xây dựng trên môi trường Node.js kết hợp framework Express.js để quản lý các dịch vụ API, cơ sở dữ liệu và giao thức truyền thông.

- Khởi chạy máy chủ Node.js/Express: Khởi tạo server tại cổng 5000, tích hợp Middleware cấu hình CORS và phân giải JSON. Hệ thống đồng thời khởi chạy một máy chủ WebSocket (ws.js) tích hợp cơ chế Heartbeat (Ping/Pong 30 giây) để giám sát và duy trì kết nối ổn định với trình duyệt của người dùng đồng thời liên tục gửi dữ liệu nhận được từ phần cứng cho frontend mà không cần gọi API.
- Kết nối MySQL và lưu dữ liệu: Sử dụng thư viện mysql2/promise với cơ chế Connection Pool để tối ưu hóa truy vấn. Dữ liệu môi trường gửi từ ESP32 được bóc tách và lưu tự động vào bảng sensor_data kèm theo mốc thời gian (timestamp).

- Xử lý API (RESTful): Xây dựng các Router và Controller phân lớp rõ ràng để cung cấp dữ liệu cho Frontend:
    - Gửi dữ liệu biểu đồ và danh sách thiết bị cho trang Tổng quan.
    - Hỗ trợ truy vấn lịch sử điều khiển (action_logs) và lịch sử cảm biến với khả năng phân trang, lọc tìm kiếm động an toàn (chống SQL Injection).
- Kết nối MQTT Broker: Tích hợp thư viện mqtt.js để làm cầu nối trung gian. Khi nhận được yêu cầu bật/tắt thiết bị qua API, Backend đẩy lệnh xuống MQTT Broker và thiết lập một bộ đệm Promise có thời gian chờ (Timeout) tối đa 3 giây (ACK_TIMEOUT_MS). Nếu nhận được phản hồi từ ESP32 trước 3 giây, API trả về "SUCCESS"; ngược lại, hệ thống trả về lỗi 504 (Timeout) và ghi nhận "FAILED" vào nhật ký.

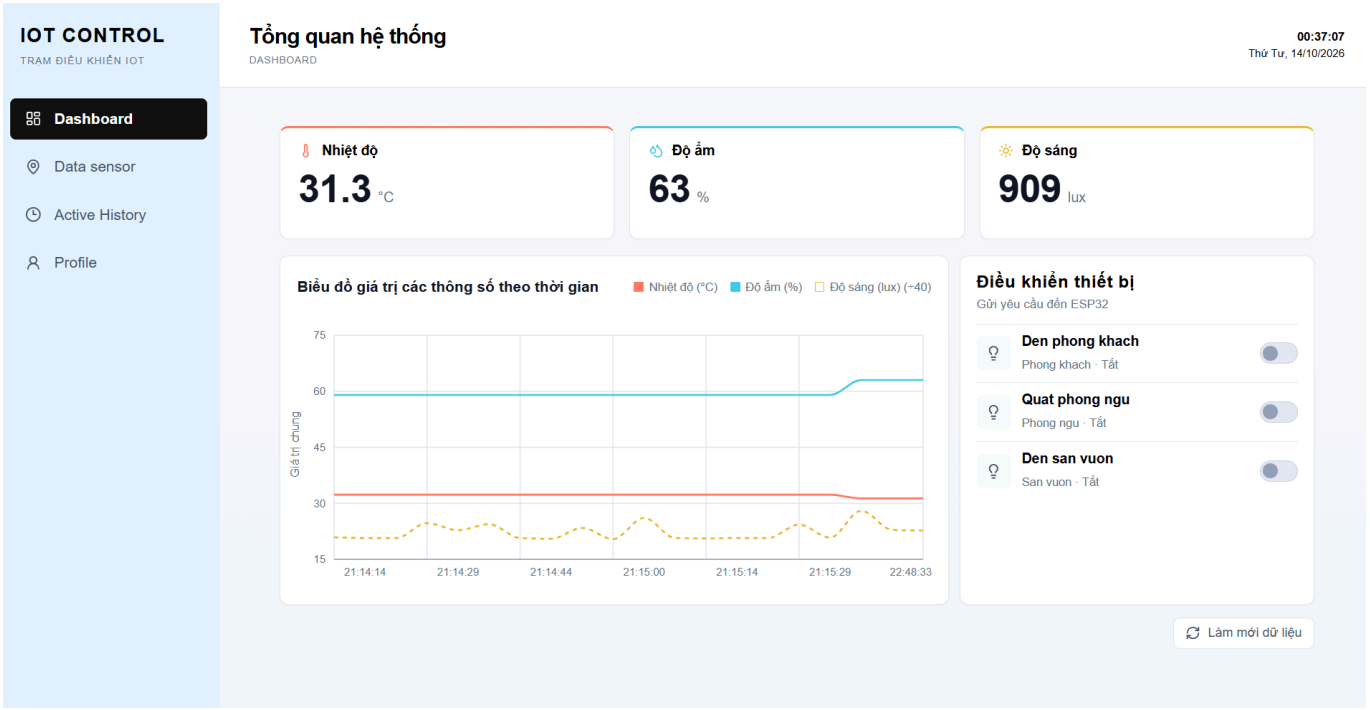
4.4. Triển khai giao diện người dùng Frontrend

Hệ thống Web được thiết kế theo dạng Single Page Application (SPA), bố cục gồm thanh điều hướng (Sidebar), thanh tiêu đề (Header) và vùng nội dung chính. Khởi chạy bằng Vite (cổng 5173), kiểm thử trực tiếp trên các trình duyệt web hiện đại (Google Chrome, Brave). Kết quả thực thi các chức năng cụ thể như sau:

a. Giao diện Tổng quan hệ thống (Dashboard)

Trang Tổng quan đóng vai trò trung tâm giám sát và điều khiển thời gian thực, gồm 3 khu vực chính:

1. Thẻ giá trị cảm biến hiện tại: Hiển thị số liệu mới nhất về Nhiệt độ (°C), Độ ẩm (%) và Độ sáng (lux). Dữ liệu được làm mới tức thì thông qua kết nối WebSocket mà không cần tải lại trang.
2. Biểu đồ theo thời gian: Sử dụng thư viện Recharts vẽ biểu đồ đường (Line Chart) biểu diễn biến thiên của 3 thông số. Để tối ưu hóa góc nhìn, giá trị độ sáng được tự động chia cho hệ số 40 (÷40) nhằm hiển thị hài hòa trên cùng một trục tọa độ với nhiệt độ và độ ẩm.
3. Bảng điều khiển thiết bị: Hiển thị danh sách các thiết bị (Đèn 1, Đèn 2, Đèn 3) kèm công tắc (Toggle switch). Khi người dùng nhấn nút, giao diện chuyển sang trạng thái "Đang chờ..." (Busy) nhằm ngăn chặn spam lệnh, cho đến khi nhận được xác nhận từ ESP32 trả về.

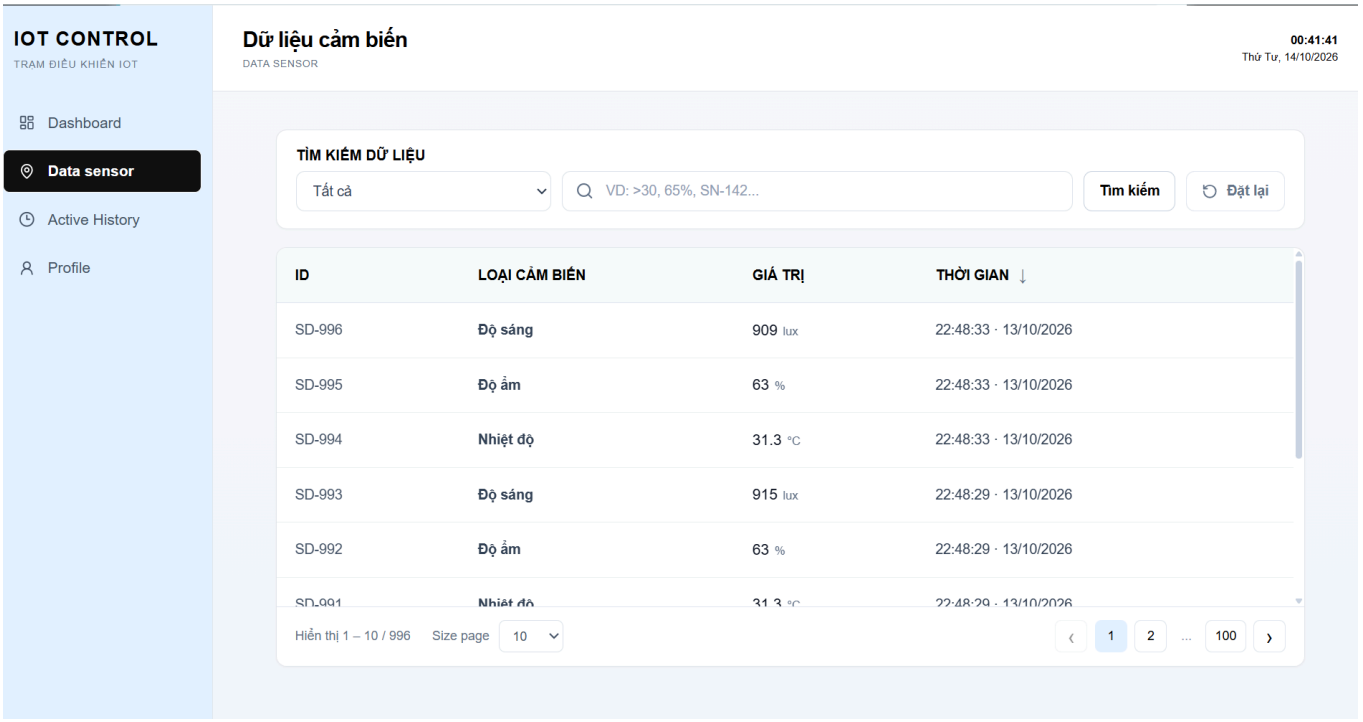


Hình 4.2 Kết quả giao diện trang Dashboard

b. Giao diện Dữ liệu cảm biến (Data Sensor)

Cung cấp chế độ xem dạng bảng chi tiết toàn bộ lịch sử đo lường được lưu trữ trong cơ sở dữ liệu.

- Khả năng phân trang: Hỗ trợ điều chỉnh kích thước trang hiển thị (5, 10, 20, 50 bản ghi/trang).
- Bộ lọc thông minh: Hỗ trợ lọc theo từng loại cảm biến cụ thể. Đặc biệt, ô tìm kiếm được lập trình bắt cú pháp toán tử (Ví dụ: nhập > 30 đối với thông số nhiệt độ) hoặc nhập chuỗi thời gian để tra cứu một cách linh hoạt.

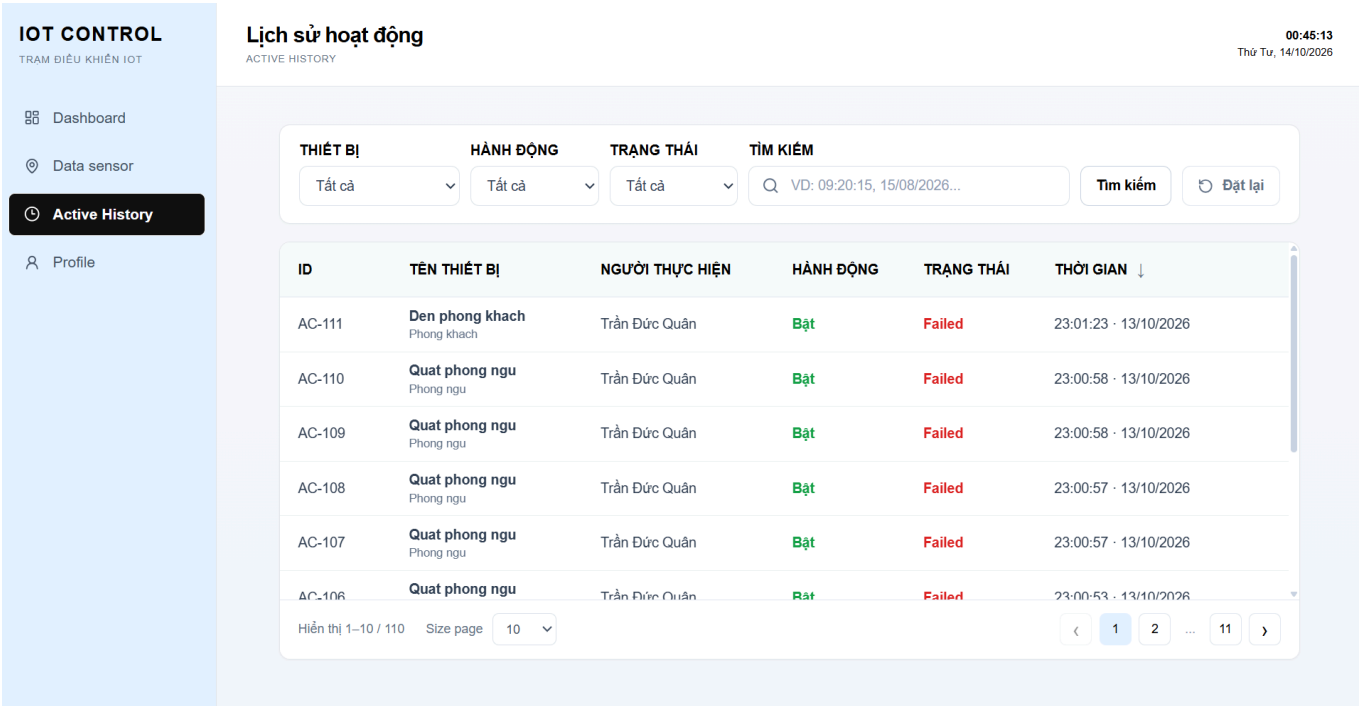


Hình 4.3 Kết quả giao diện trang Data sensor

c. Giao diện Lịch sử hoạt động (Active History)

Lưu vết (Audit Log) toàn bộ các lệnh điều khiển được gửi đến thiết bị vật lý, hỗ trợ theo dõi các hoạt động của thiết bị.

- Hiển thị chi tiết: Mã ID, Tên thiết bị, Người thực hiện, Hành động (Bật/Tắt), Trạng thái (Thành công/Lỗi) và Thời gian chuẩn xác.
- Trang bị đầy đủ các dropdown lọc (Filter) theo thiết bị, theo hành động và tìm kiếm đa năng bằng từ khóa. Cột thời gian hỗ trợ đảo chiều sắp xếp (cũ nhất/mới nhất).

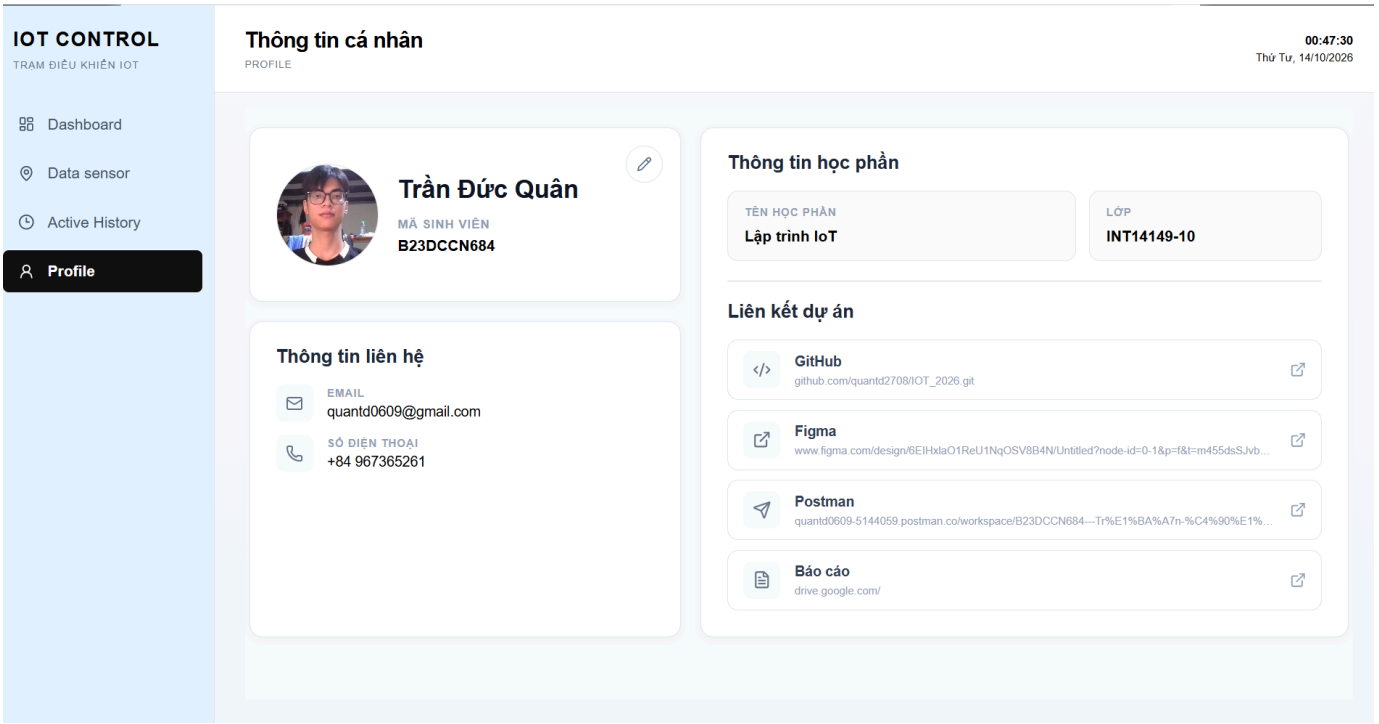


Hình 4.4 Kết quả giao diện trang Active History

d. Giao diện Thông tin cá nhân (Profile)

Nơi quản lý hồ sơ sinh viên định danh trên hệ thống.

- Cho phép xem và chỉnh sửa thông tin liên hệ (Email, Số điện thoại), thông tin học phần, cũng như lưu trữ các liên kết tài nguyên của dự án (GitHub, Figma, Postman, Báo cáo).
- Tích hợp tính năng tải lên (Upload) và xóa ảnh đại diện. Hệ thống có cơ chế kiểm tra định dạng và giới hạn dung lượng tệp (tối đa 2MB) trước khi lưu trực tiếp vào ổ cứng máy chủ.



Hình 4.5 Kết quả giao diện trang Profile

4.5. Kiểm thử và đánh giá kết quả

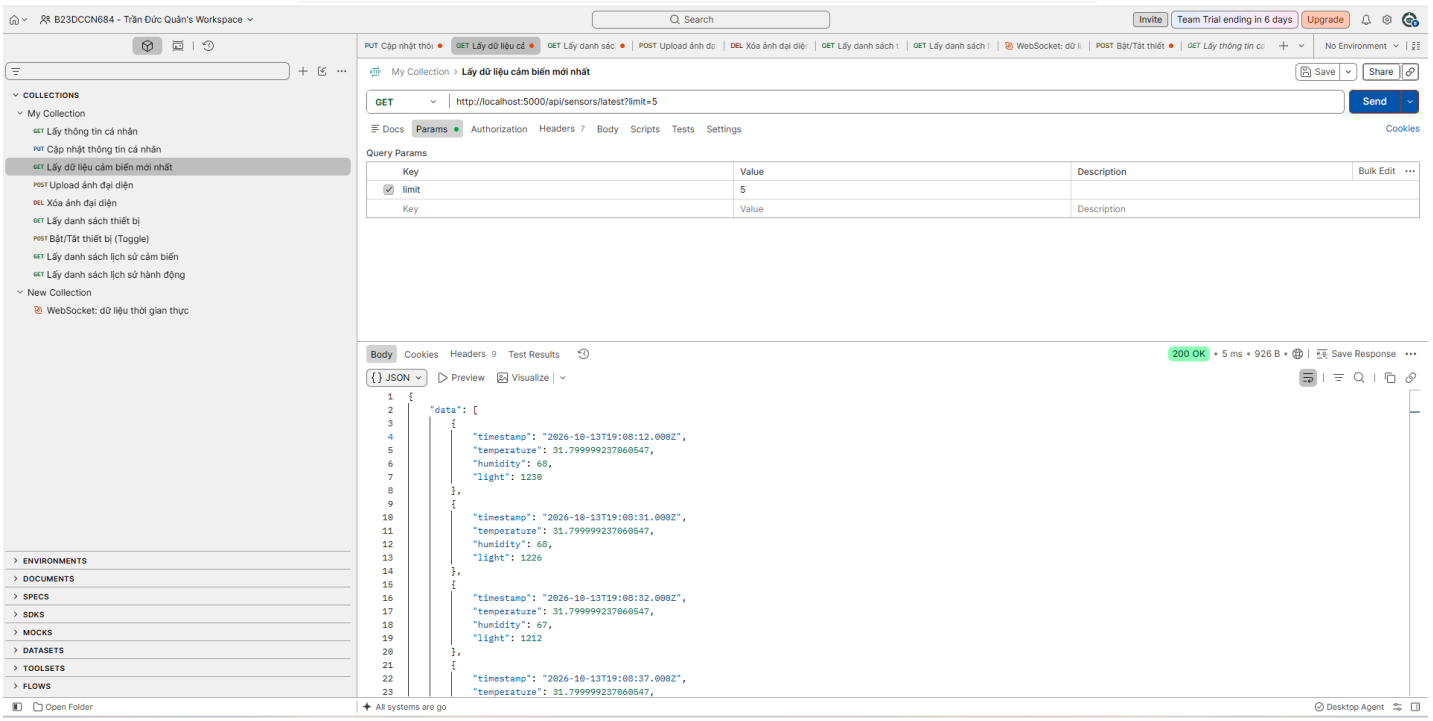
a. Kiểm thử API Backend với Postman

Trước khi ghép nối với giao diện React, toàn bộ các điểm cuối API (Endpoints) trên máy chủ Node.js đã được kiểm thử độc lập thông qua công cụ Postman để kiểm tra tính chính xác của dữ liệu JSON và các mã trạng thái HTTP (Status Codes).

URL Cơ sở (Base URL)**:** http://localhost:5000/api

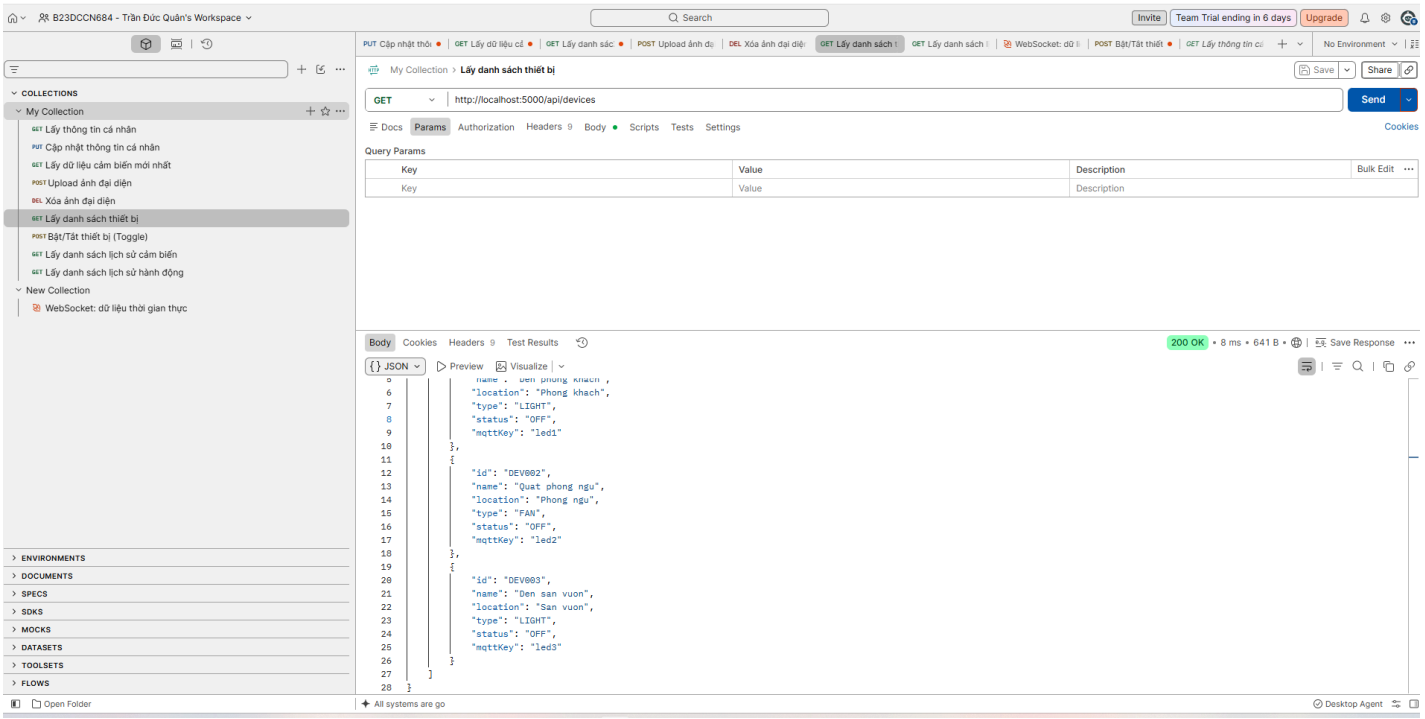
- Lấy dữ liệu cảm biến mới nhất

- Method: GET
- Endpoint: /sensors/latest
- Dữ liệu đầu vào / Query:
    - limit=5
- Status Code mong đợi: 200 OK
- Kết quả kiểm thử: Mảng JSON chứa 5 bản ghi gần nhất có đủ temperature, humidity, light



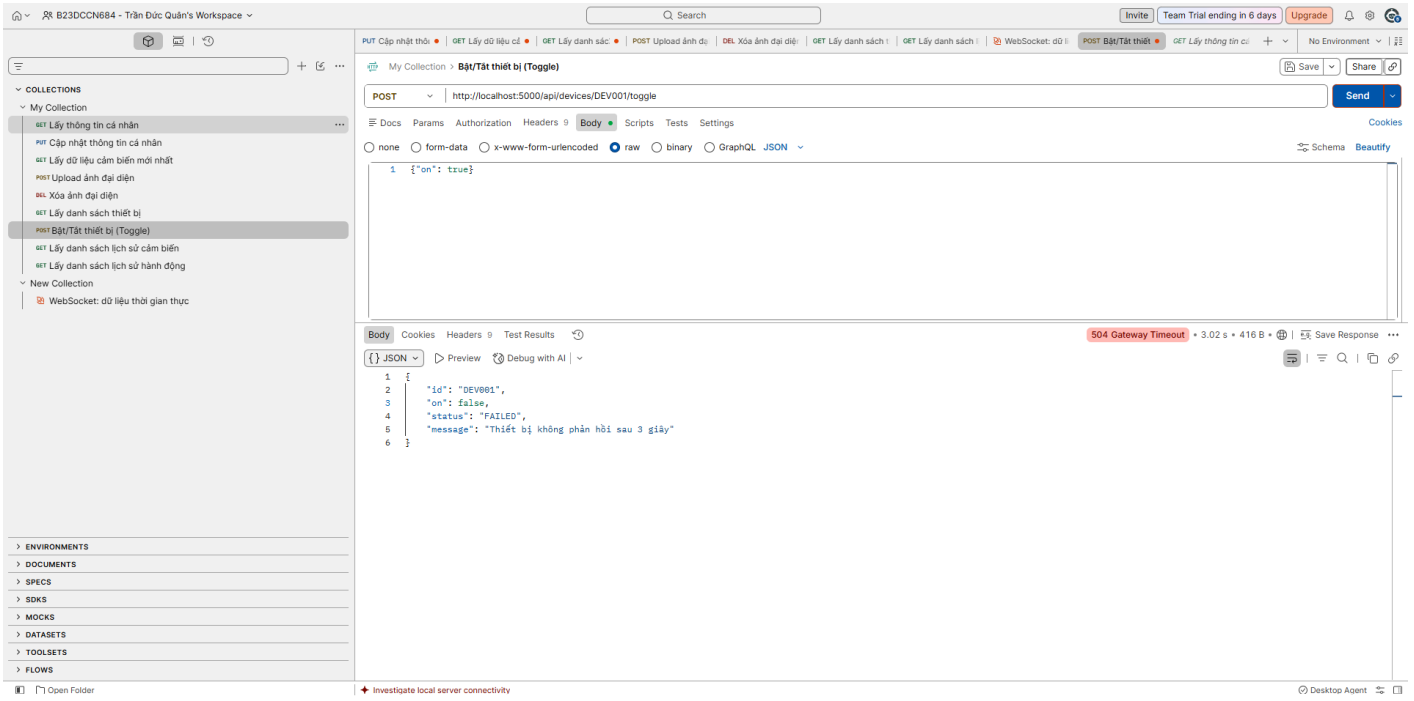
Hình 4.6 Kết quả kiểm thử lấy 5 dữ liệu mới nhất

- Lấy danh sách thiết bị
  - Method: GET
  - Endpoint:/devices
  - Dữ liệu đầu vào / Query: Không
  - Status Code mong đợi: 200 OK
  - Kết quả kiểm thử: Danh sách các thiết bị kèm id, name, status, mqttKey



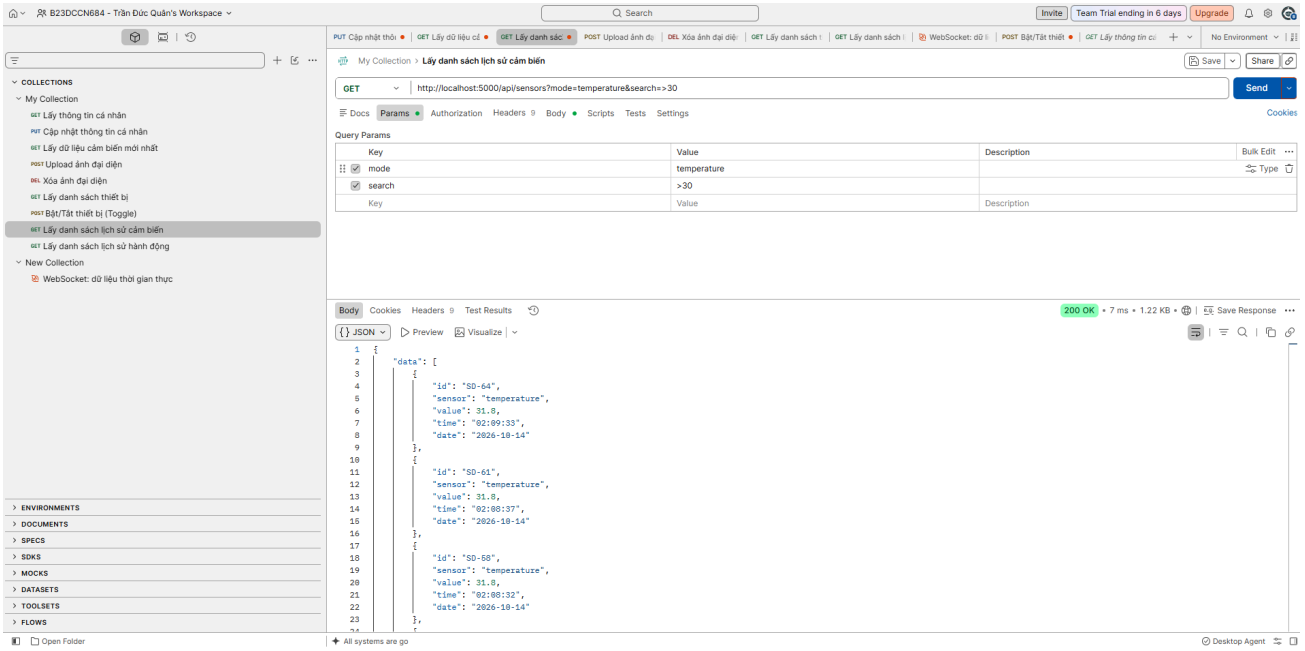
Hình 4.7 Kết quả kiểm thử lấy danh sách thiết bị

- Điều khiển bật thiết bị khi ngắt kết nối ESP32
  - Method: POST
  - Endpoint:devices/DEV001/toggle
  - Dữ liệu đầu vào / Query : Body: {"on": true}
  - Status Code mong đợi: 504 Gateway Timeout
  - Kết quả kiểm thử: Không nhận được phản hồi ACK từ ESP32 trong 3s, trả về status: "FAILED",và message: "Thiết bị không phản hồi sau 3 giây"



Hình 4.8 Kết quả kiểm thử bật thiết bị khi ngắt kết nối ESP32

- Lọc dữ liệu cảm biến
  - Method: GET
  - Endpoint: /sensors
  - Dữ liệu đầu vào / Query :
    - mode = temperature
    - search > 30
  - Status Code mong đợi: 200 OK
  - Kết quả kiểm thử: Trả về các bản ghi nhiệt độ lớn hơn 30 kèm thông tin phân trang



Hình 4.9 Kết quả kiểm thử lấy lọc dữ liệu nhiệt độ > 30

- Lọc dữ liệu cảm biến
  - Method: PUT
  - Endpoint: /profile
  - Dữ liệu đầu vào / Query :

body:{

"name": "",

"studentId": "B23DCCN684",

"email": "quantd0609@gmail.com",

"phone": "+84 967365261",

"course": "Lập trình IoT",

"className": "INT14149-10",

"github": "https://github.com/quantd2708/IOT_2026.git",

"figma": "https://www.figma.com/",

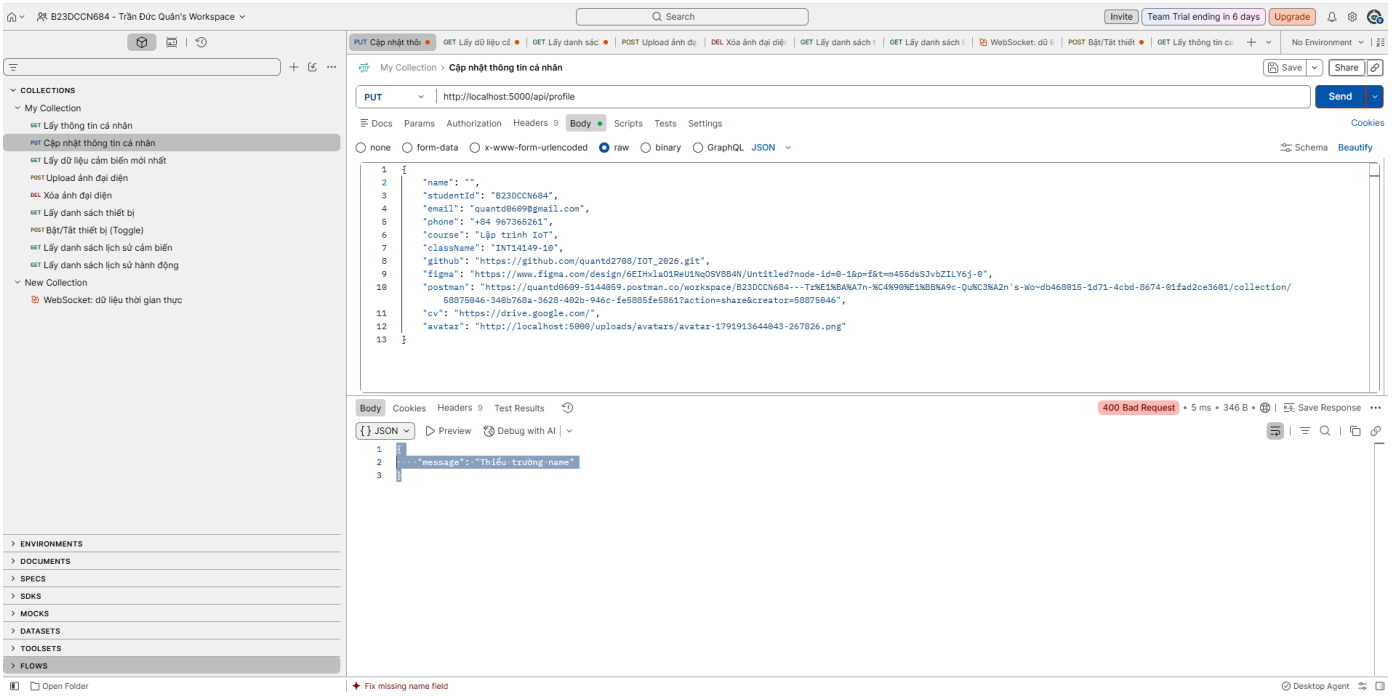
"postman": "https://quantd0609-5144059.postman.co/",

"cv": "https://drive.google.com/",

"avatar": "http://localhost:5000/uploads/avatars/avatar"

}

  - Status Code mong đợi: 400 Bad Request
  - Kết quả kiểm thử: Trả về { "message": "Thiếu trường name"}



Hình 4.10 Kết quả kiểm thử đổi xóa thông tin tên

b. Kiểm thử tích hợp toàn hệ thống

Sau khi đảm bảo các API Backend phản hồi chính xác, quá trình kiểm thử tích hợp được tiến hành nhằm đánh giá toàn bộ luồng luân chuyển dữ liệu từ phần cứng (ESP32) ↔ MQTT Broker ↔ Máy chủ Node.js ↔ Giao diện người dùng React. Mục tiêu của pha kiểm thử này là đánh giá độ trễ thực tế, sự đồng bộ dữ liệu giữa các phân hệ và trải nghiệm người dùng khi phát sinh lỗi ngoại lệ. Kết quả kiểm thử được tổng hợp tại bảng dưới đây:

STT	Luồng nghiệp vụ / Kịch bản	Kết quả mong đợi	Kết quả thực tế	Trạng thái
1	Hiển thị dữ liệu cảm biến thời gian thực (ESP32 truyền dữ liệu định kỳ)	Trên trang Dashboard, biểu đồ và các thẻ giá trị nhiệt độ, độ ẩm, độ sáng được tự động làm mới ngay khi có gói tin từ phần cứng gửi lên mà không cần tải lại trang.	Dữ liệu truyền tải qua WebSocket hiển thị trên UI gần như tức thời (trễ <1 giây) so với thời điểm ESP32 gửi lệnh client.publish.	Đạt
2	Điều khiển thiết bị (Bấm nút bật/tắt Đèn trên Dashboard)	Nút công tắc chuyển sang trạng thái "Đang chờ...". Đèn vật lý sáng/tắt lập tức. Sau đó, giao diện UI báo thành công và công tắc chuyển màu. Nhật ký được lưu vào Active History với trạng thái SUCCESS.	Mạch ESP32 kích hoạt chân GPIO thành công. Giao diện Web ngừng trạng thái bận và cập nhật màu chính xác nhờ cơ chế giải phóng Timeout tại backend khi nhận được ACK.	Đạt
3	Quản lý tương tranh điều khiển (Người dùng spam nút Bật/Tắt liên tục)	Hệ thống khóa giao diện UI (nút bị vô hiệu hóa disabled={busy}) ở Frontend, đồng thời Backend từ chối nhận lệnh mới trả về mã lỗi 409 nếu lệnh trước đó chưa hoàn thành.	Nhờ cơ chế vô hiệu hóa nút bấm (disabled={busy}) trên giao diện và độ trễ phản hồi MQTT cực thấp, hệ thống xử lý mượt mà các thao tác nhấn liên tục. Không xảy ra tình trạng treo giao diện hay treo vi điều khiển ESP32.	Đạt

4	Điều khiển khi thiết bị lỗi/mất mạng (Ngắt điện ESP32 và cố gắng bật Đèn)	Giao diện Dashboard hiển thị thông báo lỗi màu đỏ (Timeout/Lỗi gửi lệnh). Hành động được ghi vào trang Active History với trạng thái FAILED.	Sau 3 giây đếm ngược, Node.js trả về lỗi 504. Giao diện hiển thị cảnh báo "Thiết bị không phản hồi..." và trạng thái đèn được reset về ban đầu.	Đạt
5	Tìm kiếm và lọc lịch sử đo lường (Tại trang Data Sensor, tìm > 30 cho Nhiệt độ)	Bảng dữ liệu lọc chính xác các bản ghi có nhiệt độ trên 30°C. Phân trang tính toán lại tổng số trang phù hợp.	API Backend bóc tách thành công toán tử bằng Regular Expression (OPERATOR_RE) và xử lý truy vấn an toàn, trả kết quả hiển thị trên bảng.	Đạt

Bảng 4.2 Bảng kết quả kiểm thử tích hợp toàn hệ thống

4.6. Tổng kết chương

Trong chương IV, đề tài đã trình bày quá trình triển khai hệ thống IoT từ việc lắp ráp phần cứng, lập trình ESP32 đến xây dựng Backend, cơ sở dữ liệu và giao diện Frontend. Các thành phần được tích hợp thông qua MQTT, API và cơ chế cập nhật dữ liệu phù hợp với kiến trúc hệ thống.

Bên cạnh đó, chương đã xây dựng các trường hợp kiểm thử nhằm đánh giá chức năng thu thập dữ liệu cảm biến, lưu trữ thông tin, điều khiển thiết bị và tra cứu lịch sử hoạt động. Kết quả kiểm thử thực tế là cơ sở để xác định mức độ đáp ứng yêu cầu của hệ thống, đồng thời nhận diện những vấn đề cần tiếp tục cải thiện.

Nhìn chung, quá trình triển khai và đánh giá giúp kiểm chứng thiết kế đã trình bày ở các chương trước, tạo cơ sở để đưa ra kết luận về kết quả đạt được, hạn chế và hướng phát triển của đề tài.

Chương V: Tổng kết và đánh giá

5.1. Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và thi công, đề tài đã hoàn thành xuất sắc các mục tiêu đặt ra ban đầu. Hệ thống được xây dựng hoàn chỉnh từ mô hình vật lý đến các phần mềm điều khiển, giám sát. Cụ thể, đề tài đã đạt được những kết quả nổi bật sau:

1. Về mặt Phần cứng và Nhúng (Hardware & Firmware):
    - Thiết kế và lắp ráp thành công mạch điều khiển trung tâm sử dụng vi điều khiển ESP32, kết nối ổn định với các cảm biến (DHT11, quang trở) và cơ cấu chấp hành (các module LED).
    - Xây dựng chương trình nhúng hiệu quả, ứng dụng cơ chế vòng lặp không đồng bộ (non-blocking) giúp thiết bị vừa đẩy dữ liệu cảm biến định kỳ, vừa lắng nghe và phản hồi lệnh điều khiển từ MQTT Broker ngay lập tức với độ trễ cực thấp.
2. Về mặt Backend và Cơ sở dữ liệu:
    - Xây dựng thành công máy chủ Node.js/Express vững chắc, đóng vai trò trạm trung chuyển dữ liệu trung tâm. Hệ thống kết hợp nhuần nhuyễn ba giao thức: HTTP (RESTful API), MQTT (giao tiếp thiết bị) và WebSocket (đẩy dữ liệu thời gian thực).
    - Thiết kế cơ sở dữ liệu MySQL tối ưu hóa với các bảng liên kết chặt chẽ (users, devices, sensors, action_logs).
    - Triển khai các cơ chế xử lý logic nâng cao: Xử lý khóa tương tranh khi có nhiều lệnh gửi cùng lúc, cơ chế kiểm tra phản hồi (Timeout 3 giây), bảo mật dữ liệu đầu vào chống SQL Injection và tích hợp Heartbeat (Ping/Pong) để quản lý kết nối WebSocket.
3. Về mặt Giao diện người dùng (Frontend):
    - Phát triển thành công ứng dụng Web SPA bằng ReactJS hiện đại, mang lại trải nghiệm người dùng (UX) mượt mà và trực quan.
    - Hoàn thiện Dashboard giám sát thời gian thực với biểu đồ động (Recharts).
    - Xây dựng hệ thống bảng tra cứu dữ liệu cảm biến và lịch sử hoạt động chuyên sâu, hỗ trợ phân trang và bộ lọc tìm kiếm linh hoạt (cho phép lọc theo toán tử như >, <, hoặc chuỗi thời gian).
    - Tích hợp trang quản lý hồ sơ cá nhân với đầy đủ chức năng cập nhật thông tin và tải lên ảnh đại diện.

5.2. Hạn chế của hệ thống

Mặc dù hệ thống đã hoạt động ổn định và đáp ứng các yêu cầu cốt lõi của một trạm điều khiển IoT, tuy nhiên do giới hạn về thời gian và nguồn lực, đề tài vẫn còn một số điểm hạn chế:

1. Bảo mật hệ thống mạng: Quá trình truyền tải dữ liệu giữa ESP32, MQTT Broker và máy chủ hiện đang sử dụng giao thức chưa mã hóa (HTTP, MQTT, WS). Điều này tiềm ẩn rủi ro lộ lọt dữ liệu nếu triển khai trên mạng Internet công cộng thay vì mạng nội bộ (Local/LAN).
2. Cơ chế xác thực và phân quyền: Hệ thống hiện tại chủ yếu tập trung vào chức năng điều khiển và định danh tĩnh (một người dùng). Chưa có hệ thống đăng nhập, đăng xuất (Authentication) bằng Token (JWT) hay phân quyền nhiều người dùng (Admin, User) phức tạp.
3. Hạn chế của thiết bị phần cứng: Các linh kiện sử dụng (như DHT11, board cắm testboard) mang tính chất mô hình học tập, độ chính xác và độ bền chưa đáp ứng được tiêu chuẩn công nghiệp (Industrial IoT).
4. Tính năng tự động hóa: Chức năng điều khiển hiện tại hoàn toàn là thủ công (Manual) thông qua thao tác nhấn nút trên giao diện. Hệ thống chưa có tính năng cài đặt ngữ cảnh tự động (Ví dụ: tự động bật đèn khi độ sáng thấp, bật quạt khi nhiệt độ cao) hay hẹn giờ theo lịch trình.

5.3. Hướng phát triển trong tương lai

Từ những kết quả đạt được và các hạn chế còn tồn tại, tôi đề xuất một số hướng phát triển để hoàn thiện và mở rộng hệ thống trong tương lai:

1. Nâng cấp bảo mật: Triển khai chứng chỉ bảo mật SSL/TLS để mã hóa toàn bộ dữ liệu truyền tải, chuyển đổi sang sử dụng các giao thức HTTPS, WSS (Secure WebSocket) và MQTTS.
2. Phát triển hệ thống tài khoản và ứng dụng di động:
    - Tích hợp hệ thống đăng nhập OAuth2/JWT để nhiều người dùng có thể cùng quản lý các thiết bị khác nhau trong nhà.
    - Phát triển thêm ứng dụng điều khiển trên thiết bị di động (Mobile App) bằng React Native hoặc Flutter để người dùng thao tác tiện lợi hơn.
3. Tích hợp tính năng tự động hóa và AI:
    - Lập trình thêm Module Automation trên Backend, cho phép người dùng tự thiết lập các kịch bản "If-This-Then-That" (Nếu... Thì...).
    - Thu thập dữ liệu cảm biến dài hạn để ứng dụng các mô hình học máy (Machine Learning) nhằm dự đoán sự thay đổi của thời tiết hoặc tự động tối ưu hóa việc sử dụng điện năng dựa trên thói quen của người dùng.
4. Hoàn thiện phần cứng: Thiết kế và gia công mạch in (PCB) chuyên dụng thay vì dùng dây cắm rời, kết hợp in 3D vỏ hộp bảo vệ để thiết bị có tính thẩm mỹ

và độ bền cao hơn, hướng tới triển khai thành một sản phẩm thương mại hoàn chỉnh.

TÀI LIỆU THAM KHẢO

[1] Kurniawan, A. (2019). *Internet of Things Projects with ESP32: Build exciting and powerful IoT projects*. Packt Publishing.

[2] Hillar, G. C. (2017). *MQTT Essentials - A Lightweight IoT Protocol*. Packt Publishing.

[3] Monk, S. (2016). *Programming Arduino: Getting Started with Sketches* (2nd ed.). McGraw-Hill Education.

[4] Brown, E. (2019). *Web Development with Node and Express: Leveraging the JavaScript Stack* (2nd ed.). O'Reilly Media.

[5] Casciaro, M., & Mammino, L. (2020). *Node.js Design Patterns: Design and implement production-grade Node.js applications using proven patterns and techniques* (3rd ed.). Packt Publishing.

[6] Banks, A., & Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Apps* (2nd ed.). O'Reilly Media.

[7] DuBois, P. (2014). *MySQL Cookbook: Solutions for Database Developers and Administrators* (3rd ed.). O'Reilly Media.

[8] Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8th ed.). Pearson. *(Sách tham khảo mạng máy tính chung, áp dụng cho phần hiểu về TCP/IP, WebSocket)*.